

Convolutional Networks

Convolutional Networks

- Also known as Convolutional Neural Networks (CNN) – LeCun 1989.
- Specialized kind of neural network for processing data that has grid like topology
 - 1 D – Time series data
 - 2 D – Image data
- Successful in practical applications such as healthcare (COVID-19 classification), Human Activity Recognition, etc.,
- Network employs a mathematical operation called **convolution**.

Convolutional Networks (Contd.,)

- Convolution is a specialized kind of linear operation.
- CNN are simply neural networks that use convolution in place of general multiplication in at least one of their layers.

The convolution Operation



x_0



x_1



x_2

- Suppose we are tracking the location of a spaceship with a laser sensor.
- Laser sensor provides a single output $x(t)$, the position of the spaceship at time t
- x and t are real-valued
- Suppose our sensor is noisy
To obtain a less noisy estimate we would like to average several measurements

The convolution Operation



x_0



x_1



x_2

- To obtain a less noise estimate we would like to average several measurements.
- More recent measurements are important so would like to take a weighted average

$$s_t = \sum_{a=0}^{\infty} x_{t-a} w_{-a} = (x * w)_t$$

The convolution Operation

$$s_t = \sum_{a=0}^6 x_{t-a} w_{-a}$$

- In practice we would only sum over a small window
- The Weight array (w) is known as the **filter**.
- We just slide over the input and compute the value of s_t based on a window around x_t

The convolution Operation

W	0.01	0.01	0.02	0.02	0.04	0.4	0.5

X	1.00	1.10	1.20	1.40	1.70	1.80	1.90	2.10	2.20	2.40	2.50	2.70

$$S_0 = \sum_{a=0}^6 x_{t-a} w_{-a} = X_0 w_0 + X_{-1} w_{-1} + X_{-2} w_{-2} + X_{-3} w_{-3} + X_{-4} w_{-4} + X_{-5} w_{-5} + X_{-6} w_{-6}$$

$$S_0 = (1.90 * 0.5) + (1.80 * 0.4) + (1.70 * 0.04) + (1.40 * 0.02) + (1.20 * 0.02) + (1.10 * 0.01) + (1.00 * 0.01)$$

S	1.811	1.96	2.11	2.16	2.28	2.42

The convolution Operation



- In the above process input is one dimensional
- How about the convolution operation on a 2D input.
- Consider the image as 2D input – now to use a 2D filter of size $(m \times n)$
- Then the 2D formula looks like

$$S_{i,j} = (I * K)_{i,j} = \sum_{a=0}^{m-1} \sum_{b=0}^{n-1} I_{i-a,j-b} K_{a,b}$$

A Toy example

Input

A	B	C	D
E	F	G	H
I	J	K	L

Kernel

W	X
Y	Z

Output

$AW+BX+EY+FZ$	$BW+CX+FY+GZ$	$CW+DX+GY+HZ$
$EW+FX+IY+JZ$	$FW+GX+JY+KZ$	$GW+HZ+KY+LZ$

A Toy example



$$* \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} =$$



$$* \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix} =$$



A Toy example

7	2	3	3	8
4	5	3	8	4
3	3	2	8	4
2	8	7	2	7
5	4	4	5	4

*

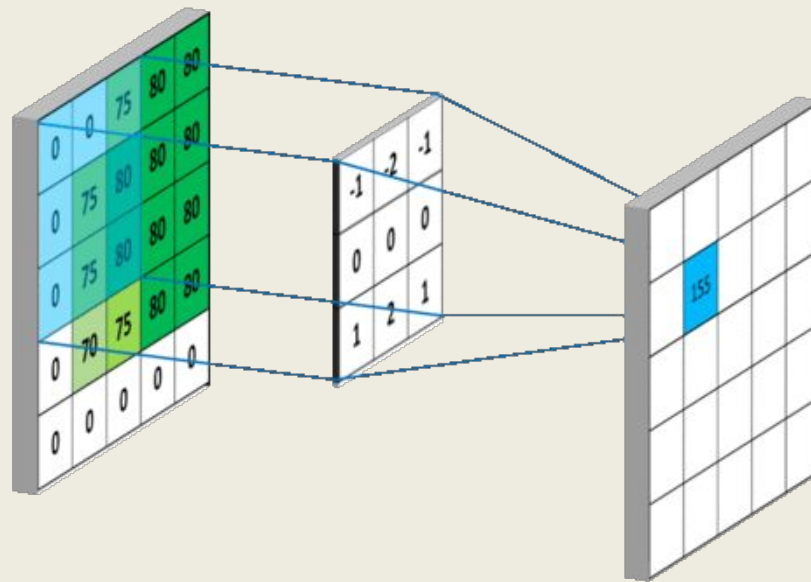
1	0	-1
1	0	-1
1	0	-1

=

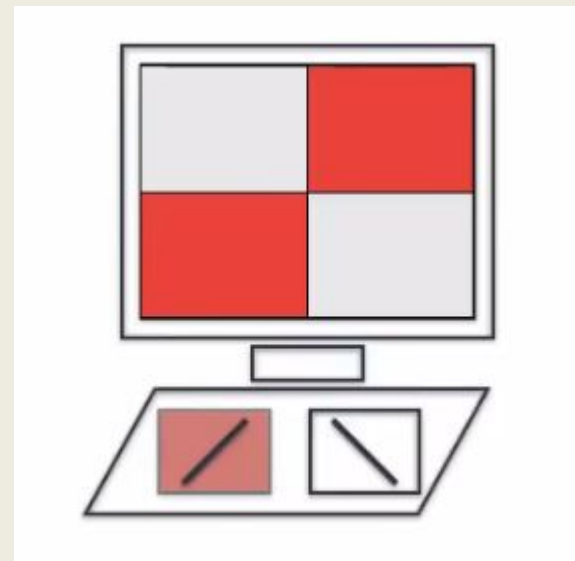
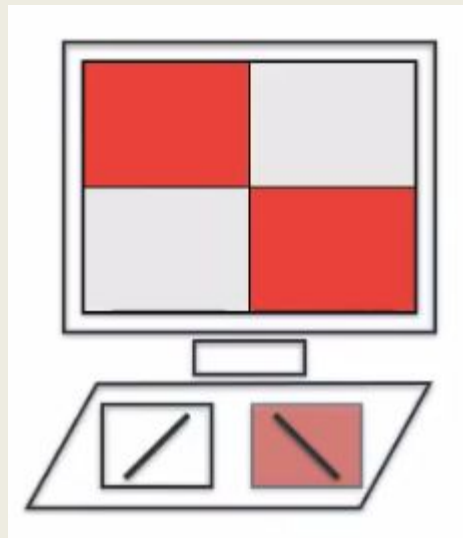
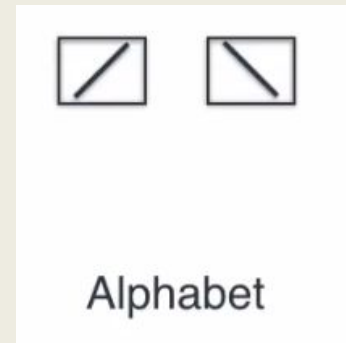
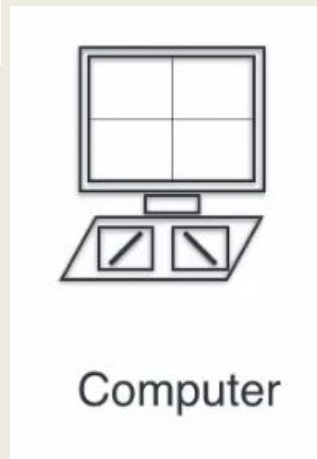
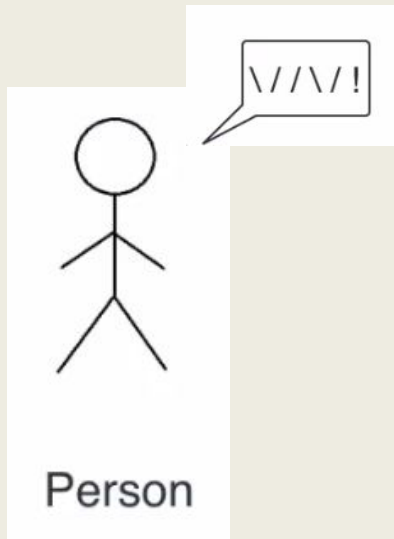
6		

$$7 \times 1 + 4 \times 1 + 3 \times 1 + 2 \times 0 + 5 \times 0 + 3 \times 0 + 3 \times -1 + 3 \times -1 + 2 \times -1 = 6$$

A Toy example

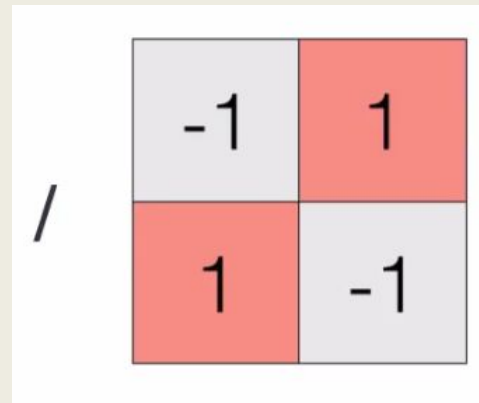
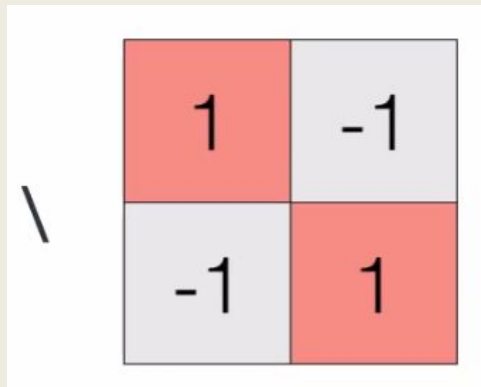
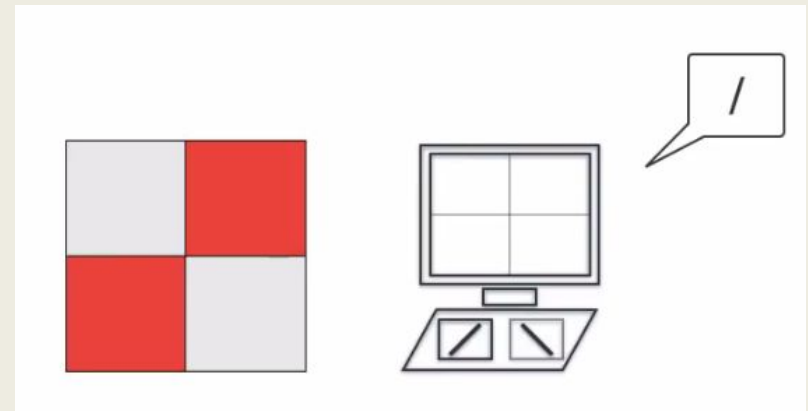
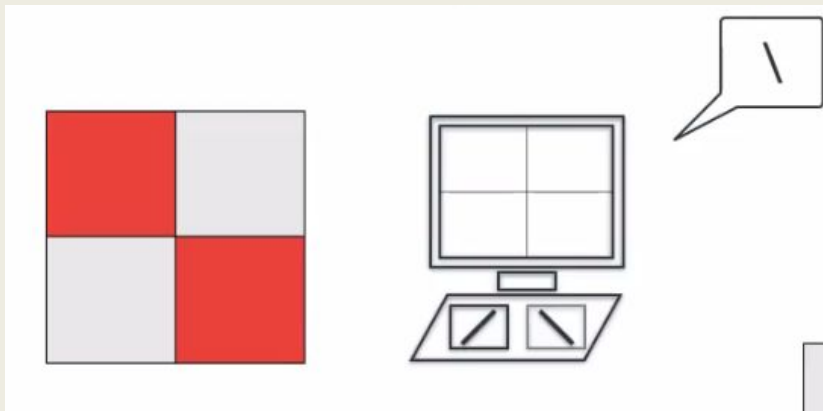


How to decide the filter ?



How to decide the filter ?

- Image recognition software



How to decide the filter ?

\

1	-1
-1	1

1	-1	-1	1
---	----	----	---

/

-1	1
1	-1

-1	1	1	-1
----	---	---	----

How to decide the filter ?

- Sum up all the values



Diagram illustrating a 1D convolution operation. The input sequence is $1 + -1 + -1 + 1$, where the first and last elements (1 and 1) are highlighted in red. The kernel sequence is $1 + -1 + -1 + 1$, where the first and last elements (1 and 1) are highlighted in red. The result of the convolution is $= 0$.

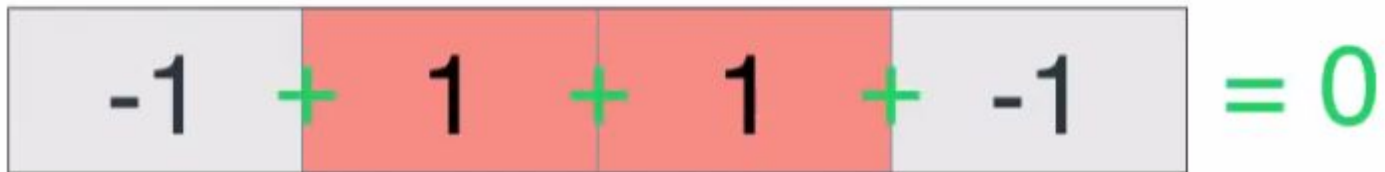


Diagram illustrating a 1D convolution operation. The input sequence is $-1 + 1 + 1 + -1$, where the second and third elements (1 and 1) are highlighted in red. The kernel sequence is $-1 + 1 + 1 + -1$, where the second and third elements (1 and 1) are highlighted in red. The result of the convolution is $= 0$.

How to decide the filter ?

- Multiply all the values

Diagram illustrating a 1D convolution operation. The input is a 1x7 grid with values 1 , x , -1 , x , -1 , x , and 1 . The first four cells (1, x, -1, x) are shaded red, and the last three cells (-1, x, 1) are shaded grey. The output is $= 1$.

Diagram illustrating a 1D convolution operation. The input is a 1x7 grid with values -1 , x , 1 , x , 1 , x , and -1 . The first cell (-1) and the last cell (-1) are shaded grey, and the middle four cells (x, 1, x, 1) are shaded red. The output is $= 1$.

How to decide the filter ?

\

$\begin{matrix} + \\ 1 \end{matrix}$	$\begin{matrix} - \\ -1 \end{matrix}$
$\begin{matrix} - \\ -1 \end{matrix}$	$\begin{matrix} + \\ 1 \end{matrix}$

$\begin{matrix} + \\ 1 \end{matrix}$	$\begin{matrix} - \\ -1 \end{matrix}$	$\begin{matrix} - \\ -1 \end{matrix}$	$\begin{matrix} + \\ 1 \end{matrix}$
--------------------------------------	---------------------------------------	---------------------------------------	--------------------------------------

/

$\begin{matrix} + \\ -1 \end{matrix}$	$\begin{matrix} - \\ 1 \end{matrix}$
$\begin{matrix} - \\ 1 \end{matrix}$	$\begin{matrix} + \\ -1 \end{matrix}$

$\begin{matrix} + \\ -1 \end{matrix}$	$\begin{matrix} - \\ 1 \end{matrix}$	$\begin{matrix} - \\ 1 \end{matrix}$	$\begin{matrix} + \\ -1 \end{matrix}$
---------------------------------------	--------------------------------------	--------------------------------------	---------------------------------------

How to decide the filter?

\

$\begin{matrix} + \\ 1 \end{matrix}$	$\begin{matrix} - \\ -1 \end{matrix}$
$\begin{matrix} - \\ -1 \end{matrix}$	$\begin{matrix} + \\ 1 \end{matrix}$

$$\begin{array}{cccc} + & - & - & + \\ \hline 1 & -1 & -1 & 1 \\ \hline +1 & +1 & +1 & +1 \end{array} = 4$$

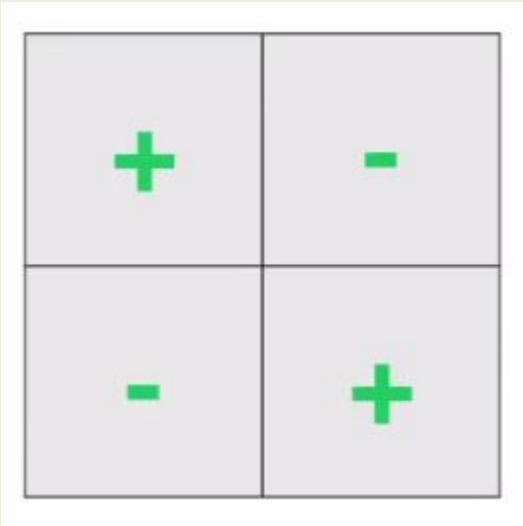
/

$\begin{matrix} + \\ -1 \end{matrix}$	$\begin{matrix} - \\ 1 \end{matrix}$
$\begin{matrix} - \\ 1 \end{matrix}$	$\begin{matrix} + \\ -1 \end{matrix}$

$$\begin{array}{cccc} + & - & - & + \\ \hline -1 & 1 & 1 & -1 \\ \hline -1 & -1 & -1 & -1 \end{array} = -4$$

How to decide the filter?

- Filter

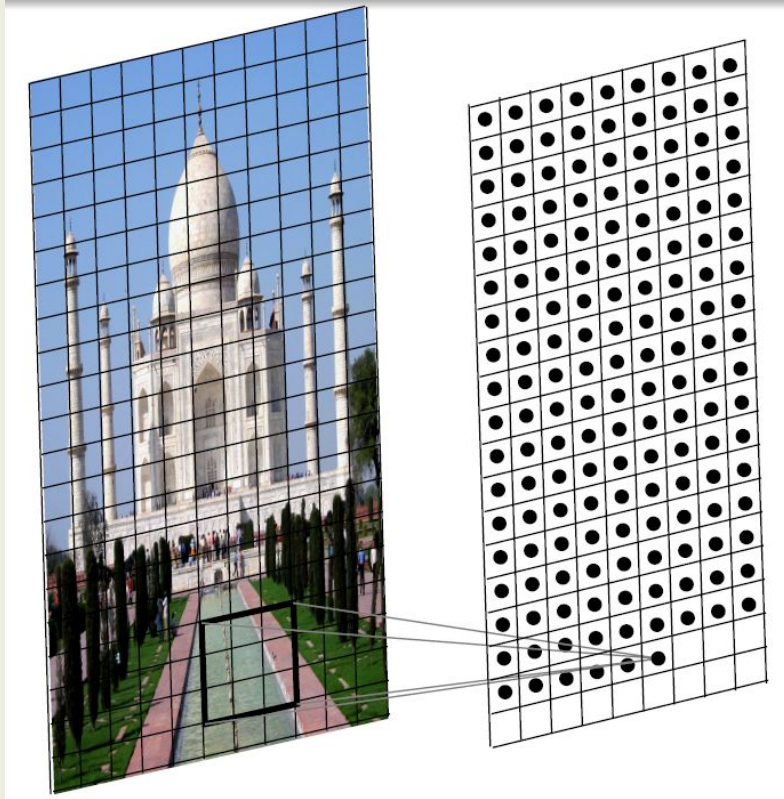


- Feature map

If positive, “\”

If negative, “/”

How to decide the filter?



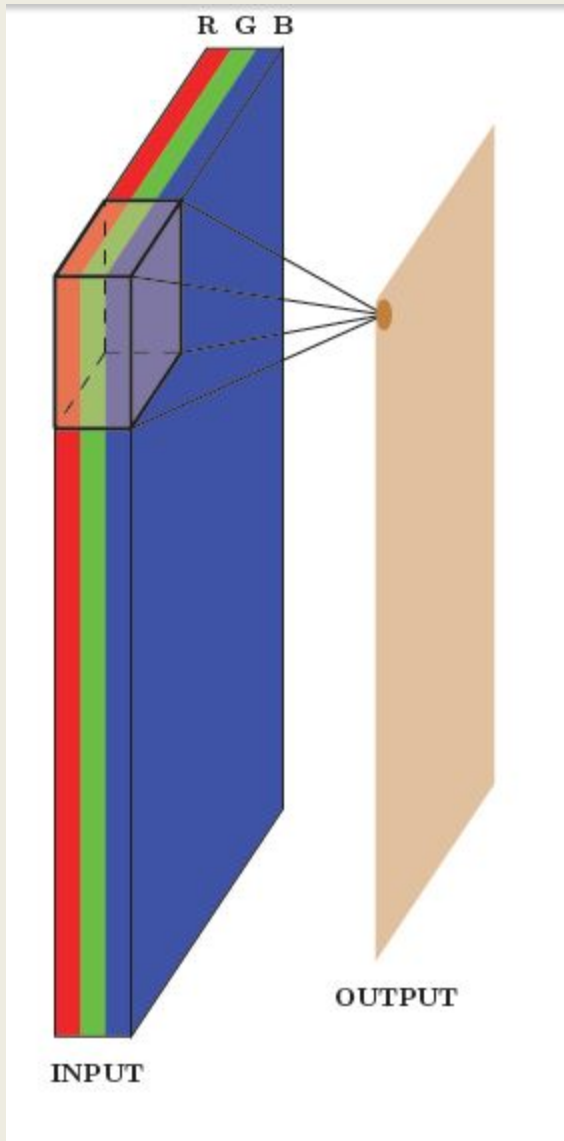
- Feature map – Each time we slide the kernel we get one value in the output.
- Resulting output is called a feature map.
- We can use multiple filters to get multiple feature maps.

Question

- In the 1-D case, we slide a one-dimensional filter over a one dimensional input
- In the 2-D case, we slide a two-dimensional filter over a two dimensional input
- In the case of 3-D data ????

3-D case

- It will be 3D and refer to it as a volume
- we will slide the volume over the 3D input and compute the convolution operation
- Actually, we are doing a 2D convolutional operations on 3D input(because the filter moves along the height and the width but not along the depth)
- As a result, the output will be 2D



3-Dimensional case ?

0	0	0	0	0	0	...
0	156	155	156	158	158	...
0	153	154	157	159	159	...
0	149	151	155	158	159	...
0	146	146	149	153	158	...
0	145	143	143	148	158	...
...	...	...	...	...	...	...

Input Channel #1 (Red)

0	0	0	0	0	0	...
0	167	166	167	169	169	...
0	164	165	168	170	170	...
0	160	162	166	169	170	...
0	156	156	159	163	168	...
0	155	153	153	158	168	...
...	...	...	...	...	...	...

Input Channel #2 (Green)

0	0	0	0	0	0	...
0	163	162	163	165	165	...
0	160	161	164	166	166	...
0	156	158	162	165	166	...
0	155	155	158	162	167	...
0	154	152	152	157	167	...
...	...	...	...	...	...	...

Input Channel #3 (Blue)

-1	-1	1
0	1	-1
0	1	1

Kernel Channel #1



308

1	0	0
1	-1	-1
1	0	-1

Kernel Channel #2



-498

0	1	1
0	1	0
1	-1	1

Kernel Channel #3



164

+ 1 = -25

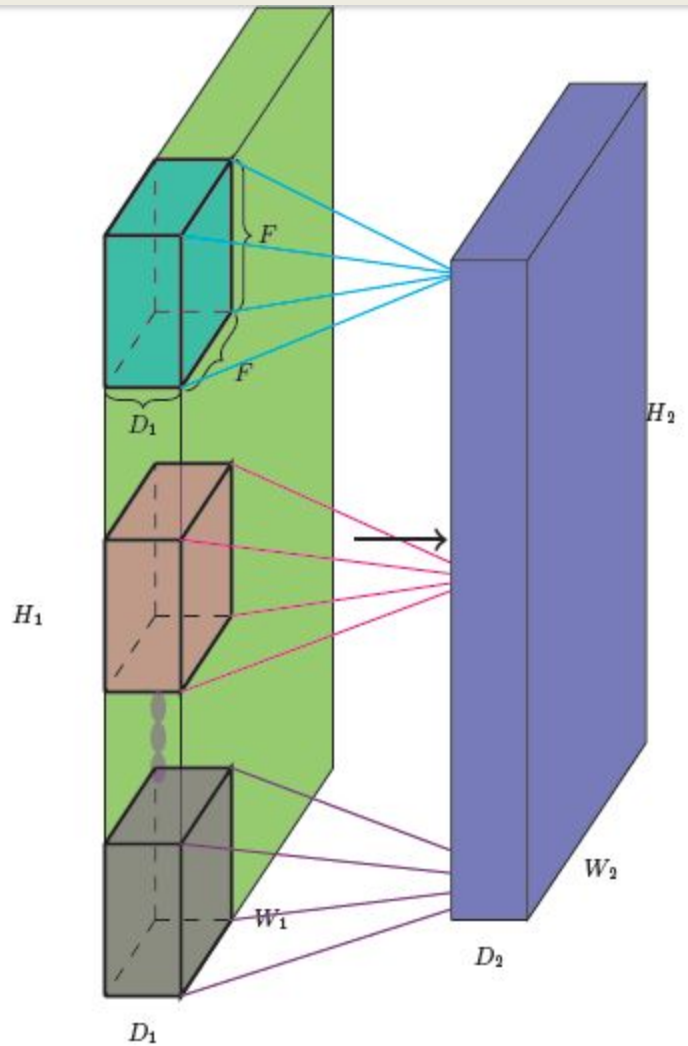


Bias = 1

Output

-25				...
				...
				...
				...
...	...	...	...	...

Relation between I/P, O/P and filters



- Width (W_1), Height (H_1) and Depth (D_1)
- Number of filters be K
- The output is

$$W_2 * H_2 * D_2$$

Relation between I/P, O/P and filters

- Compute the dimension of (W_2, H_2)

1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

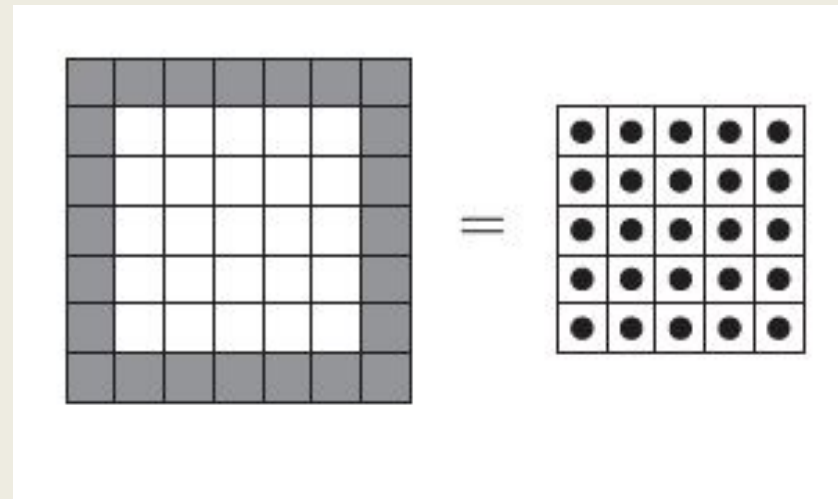
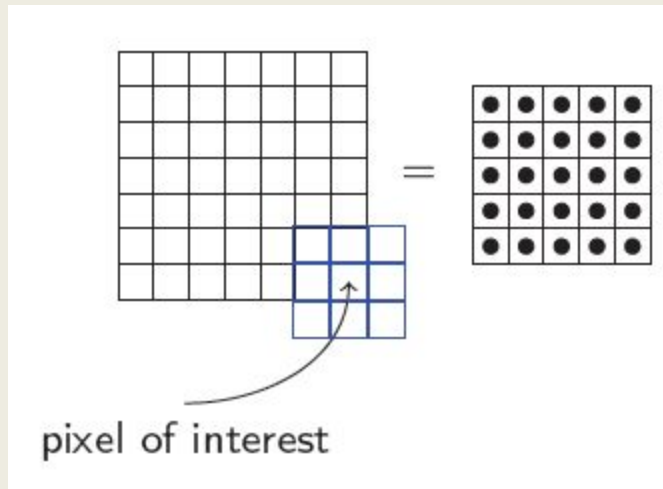
Image

4		

Convolved
Feature

Relation between I/P, O/P and filters

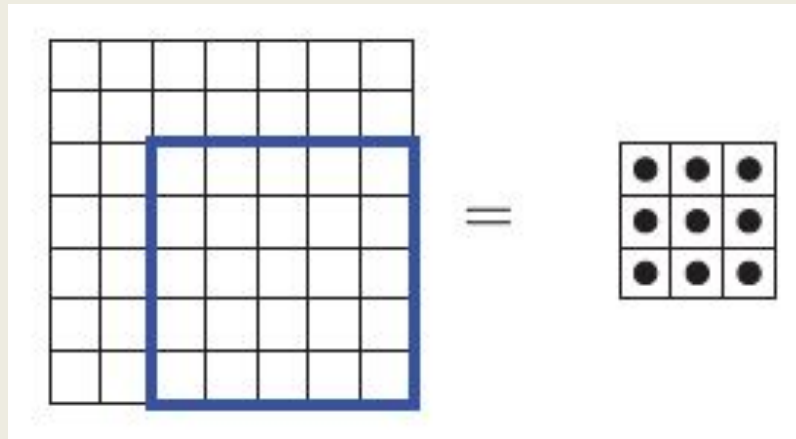
- We can't place the kernel at the corners as it will cross the boundary



- This is true for all the shaded points (the kernel crosses the input boundary)

Relation between I/P, O/P and filters

- If the kernel size increases, the output leads to even smaller dimension



$$W_2 = W_1 - F + 1$$

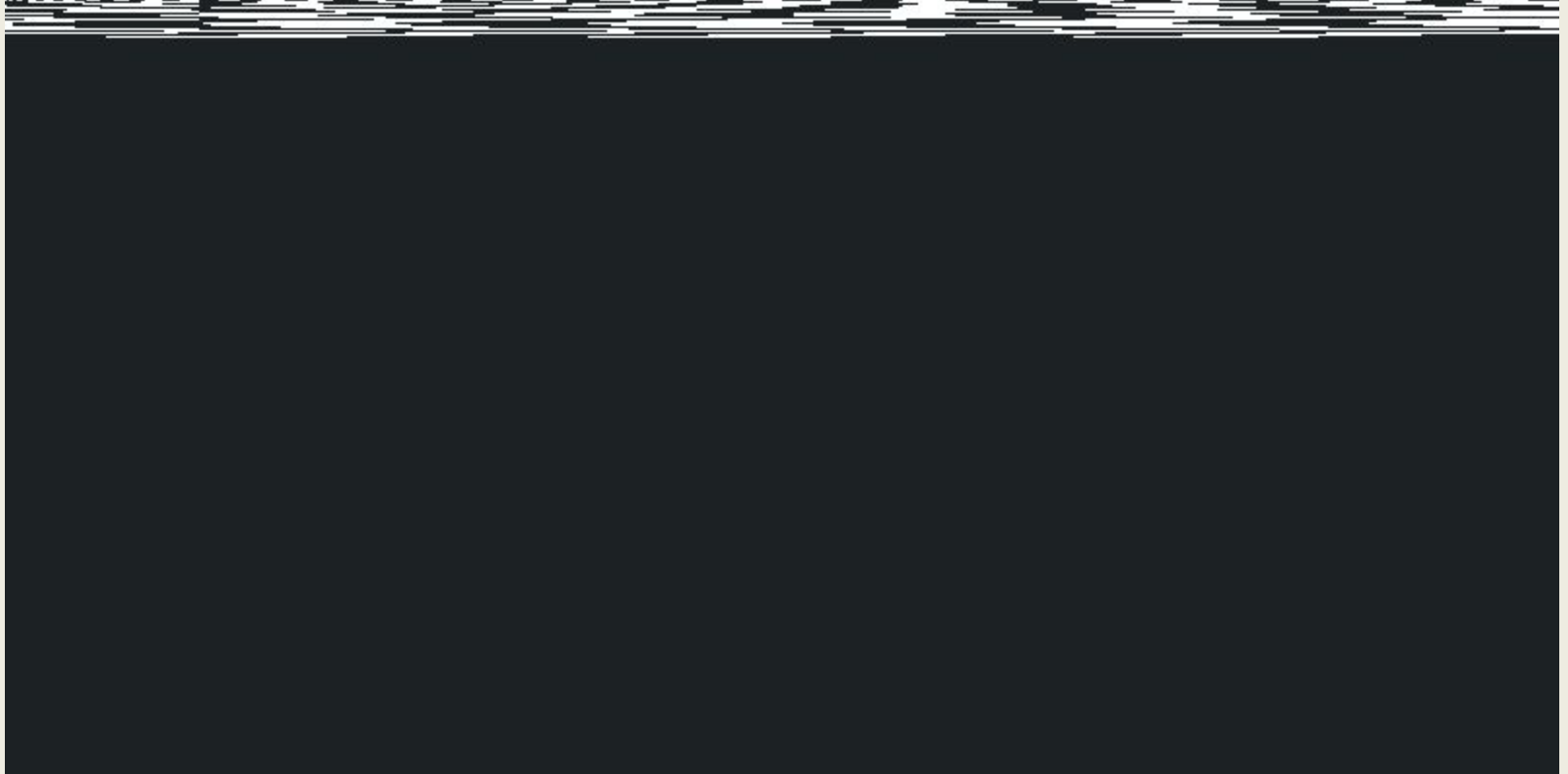
$$H_2 = H_1 - F + 1$$

Relation between I/P, O/P and filters - Padding

- What if we want the output to be of same size of the input ? - Padding
- Pad the inputs with appropriate number of 0 inputs so that you can now apply the kernel at the corners

Relation between I/P, O/P and filters - Padding

- Pad $P=I$ with 3×3 kernel i.e to add one row and one column of 0 inputs at the top, bottom, left and right



Relation between I/P, O/P and filters - Padding

- Pad $P=1$ with 3×3 kernel i.e to add one row and one column of 0 inputs at the top, bottom, left and right

$$W_2 = W_1 - F + 2P + 1$$

$$H_2 = H_1 - F + 2P + 1$$

Relation between I/P, O/P and filters - Stride

- It defines the intervals at which the filter is applied (here $S = 2$)
- Skipping every 2nd pixel which will again result in an output which is of smaller dimensions

Relation between I/P, O/P and filters - Stride

Relation between I/P, O/P and filters - Stride

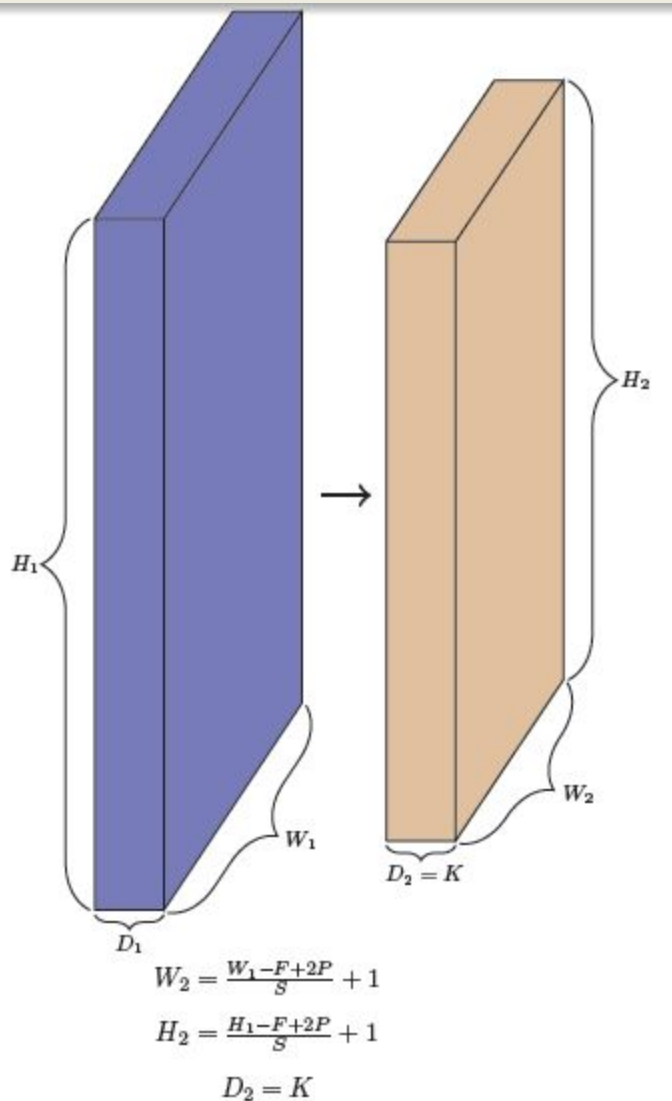
- It defines the intervals at which the filter is applied (here $S = 2$)
- Skipping every 2nd pixel which will again result in an output which is of smaller dimensions

$$W_2 = \frac{W_1 - F + 2P}{S} + 1$$

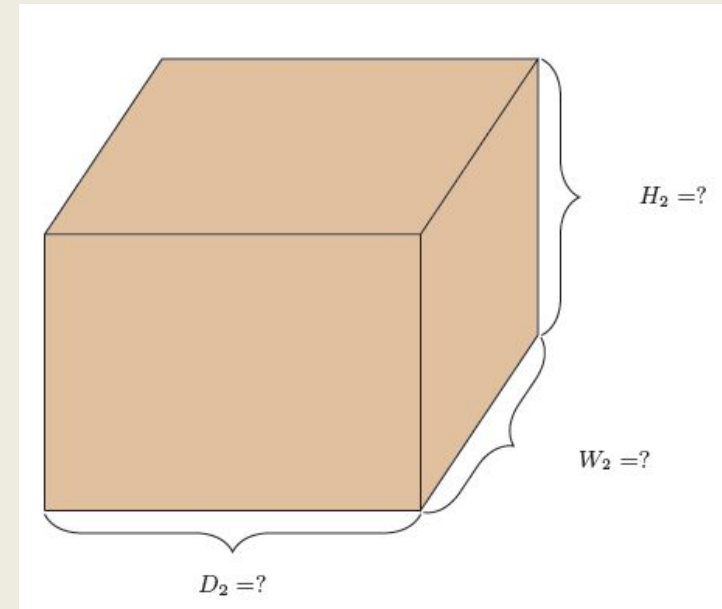
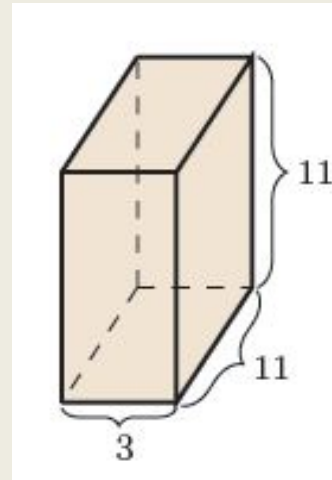
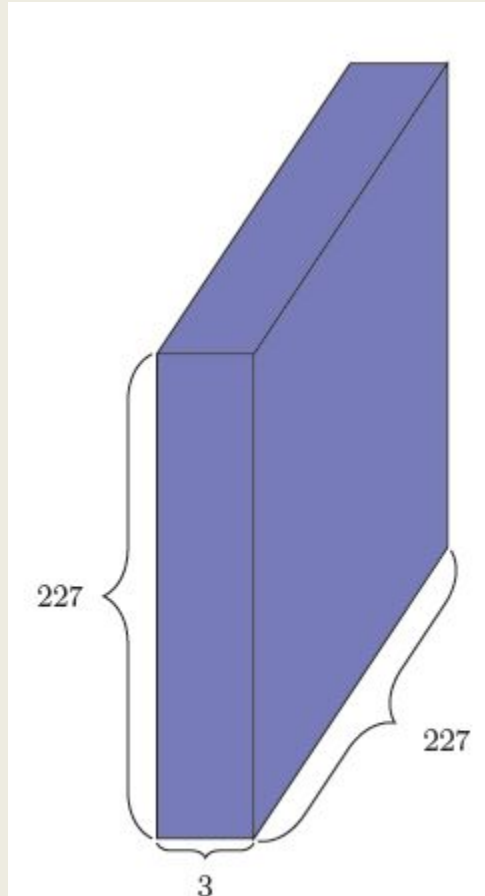
$$H_2 = \frac{H_1 - F + 2P}{S} + 1$$

Relation between I/P, O/P and filters - Depth

- Each filter gives us one 2D Output
- K filter will give K such 2D output, then the resulting output will be



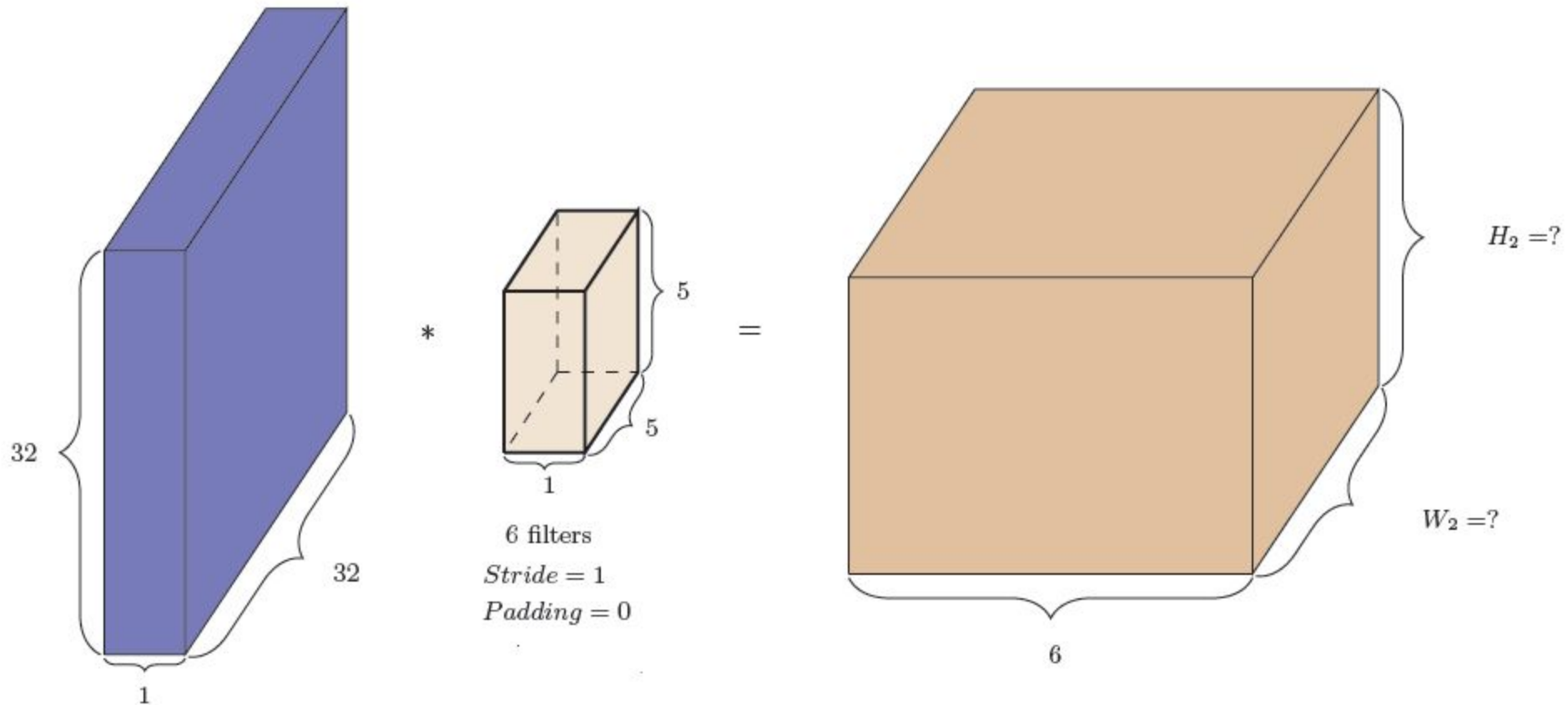
Exercise



Filters = 96
Stride = 4
Padding = 0

$D_2 = 96$
 $H_2 = 55$
 $W_2 = 55$

Exercise



$H_2 = 28$

$W_2 = 28$

Connection b/w Convolution and Neural Network.



Features

1. Bat
2. Ball
3. Stump

Connection b/w Convolution and Neural Network.



Static Feature Extraction

No Learning

Classification [Bowler,
Batsman, Wicket Keeper]

Learning Weights of Classifier

Connection b/w Convolution and Neural Network

- Instead of using handcrafted features, CNN makes us to learn kernels/ filters in addition to the weights of the classifier.



*

$$\begin{matrix} & \begin{matrix} 1 & 2 & \dots & n \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ \vdots \\ m \end{matrix} & \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ a_{31} & a_{32} & \dots & a_{3n} \\ \vdots & \vdots & \vdots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix} \end{matrix}$$

= Batsman



Learn these weights

Connection b/w Convolution and Neural Network



*

$$\begin{matrix}
 & \begin{matrix} 1 & 2 & \dots & n \end{matrix} \\
 \begin{matrix} 1 \\ 2 \\ \vdots \\ m \end{matrix} & \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix}
 \end{matrix}$$

=

Batsman

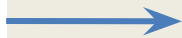


Learn these weights

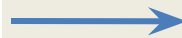
CNN allows us to learn **multiple meaningful** kernels/filters in addition to learn the weights of the classifier.

How to learn multiple meaningful kernels/ filters?

- Treating these kernels as parameters and learning them in addition to the weights of the classifier (using backpropagation)

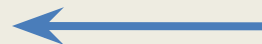


$$\begin{matrix} & \begin{matrix} 1 & 2 & \dots & n \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ \vdots \\ m \end{matrix} & \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ a_{31} & a_{32} & \dots & a_{3n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix} \end{matrix}$$



Batsman

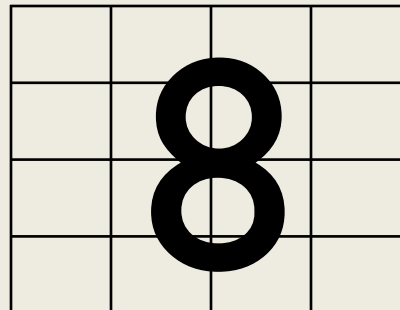
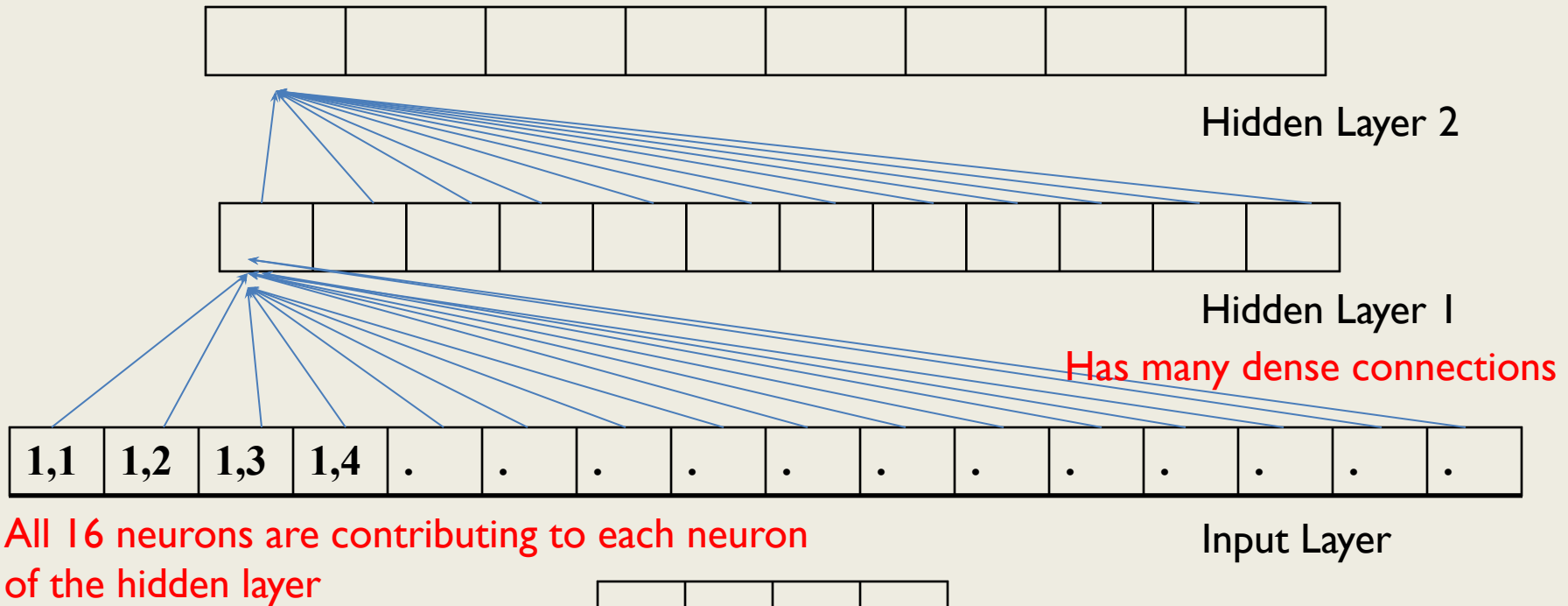
$$\begin{matrix} & \begin{matrix} 1 & 2 & \dots & n \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ \vdots \\ m \end{matrix} & \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ a_{31} & a_{32} & \dots & a_{3n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix} \end{matrix}$$



Backpropagation

How it is different from FFN?

A Regular Feed Forward Neural Network

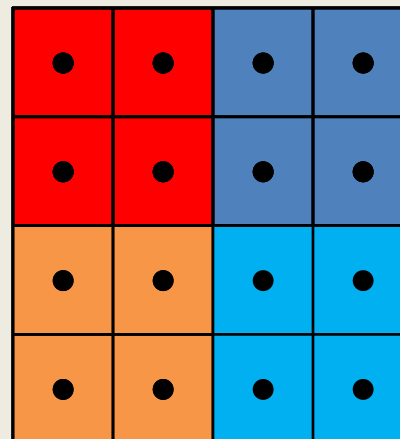


How it is different from FFN?

In case of **Convolutional Neural Network**

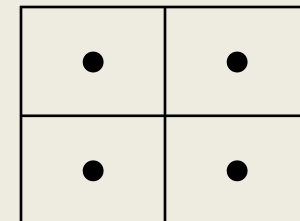


Input Image



Kernel/ Filter

*

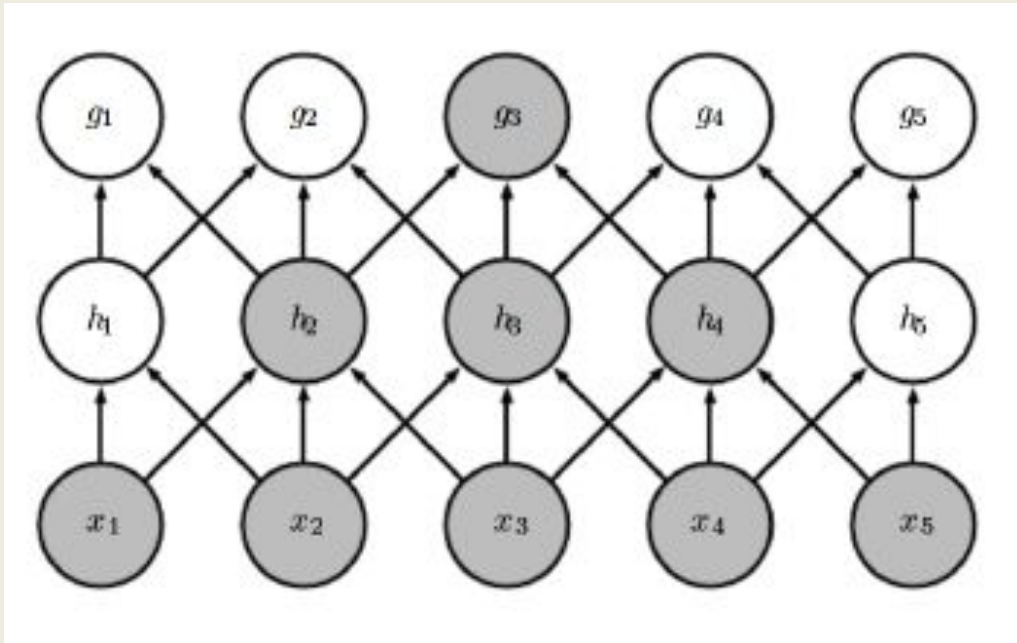


Only few neurons participate
In the process of each hidden
Layer neurons

How it is different from FFN?

- Only few neurons participate in the process of each hidden layer neurons
- For example, pixel 1,2,5 and 6 contribute to neuron 1 of hidden layer 1
- The connections are sparser
- The sparse connectivity reduces the number of parameters in the model.

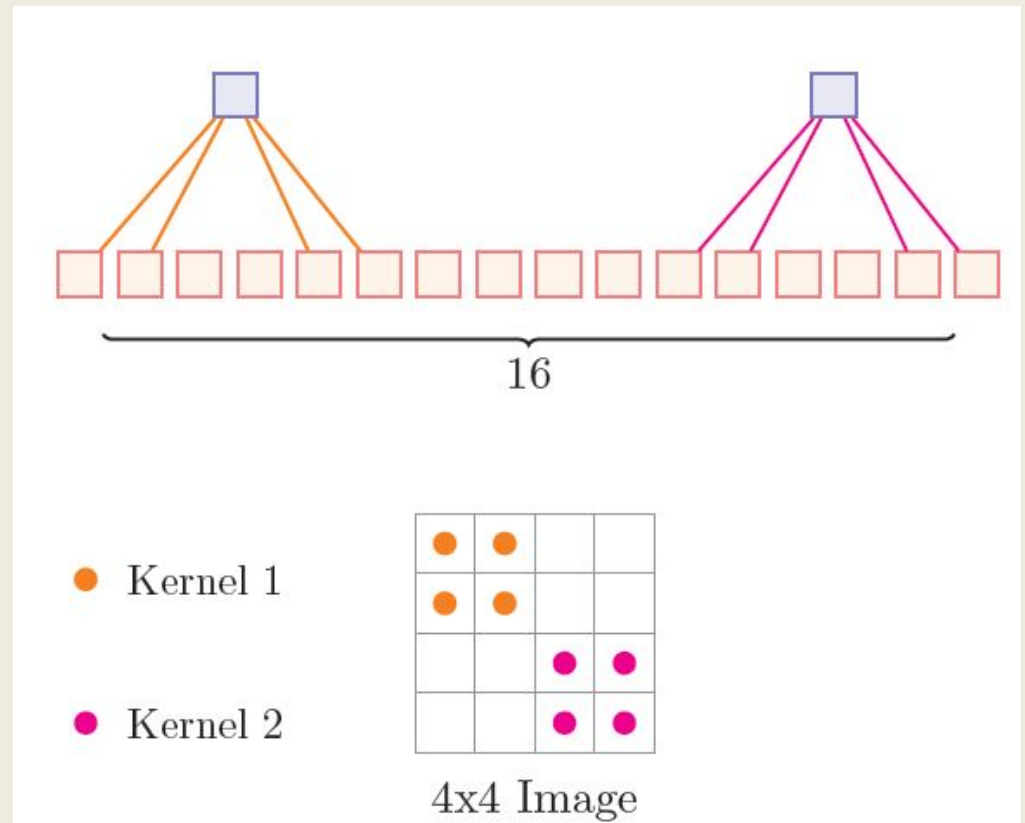
Is sparse good or not?



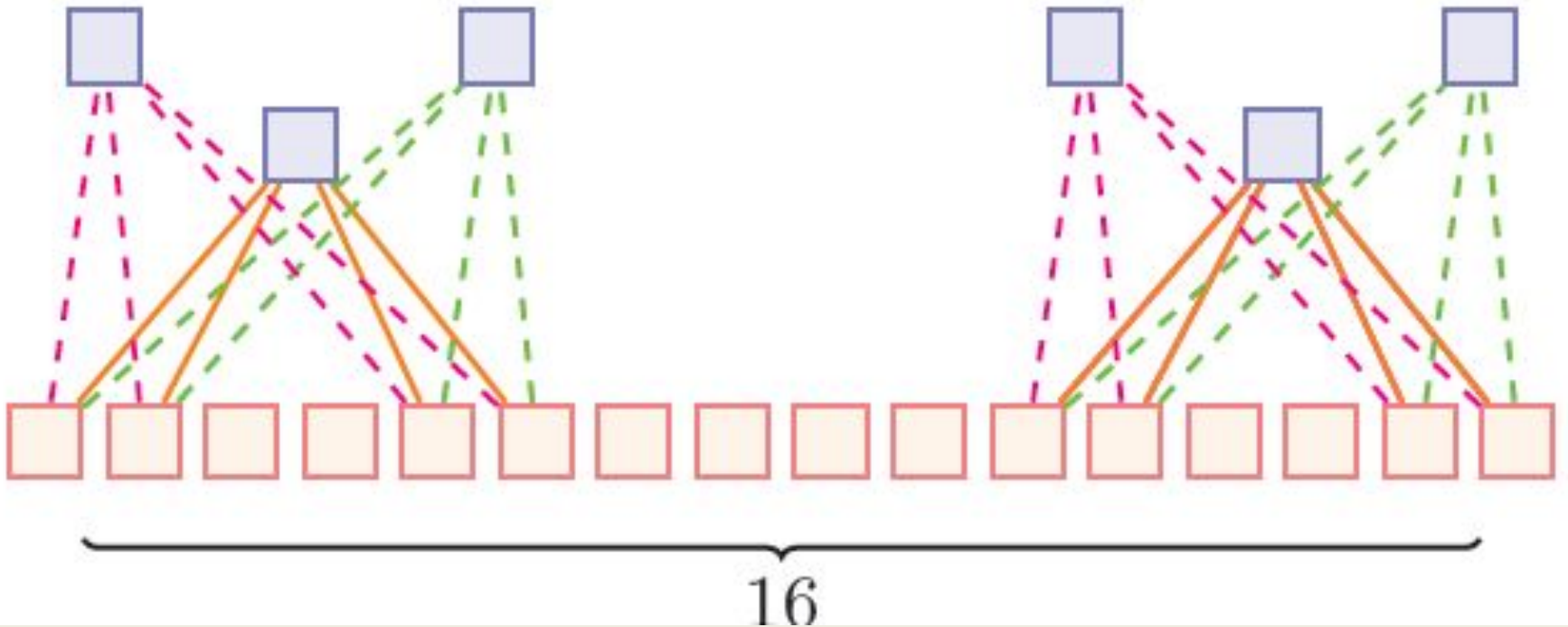
- Are we losing any information here due to sparsity??
- **No**
- Two neurons do not have any interaction in layer 1, but indirectly contribute to the operation on layer 2.

Weight sharing

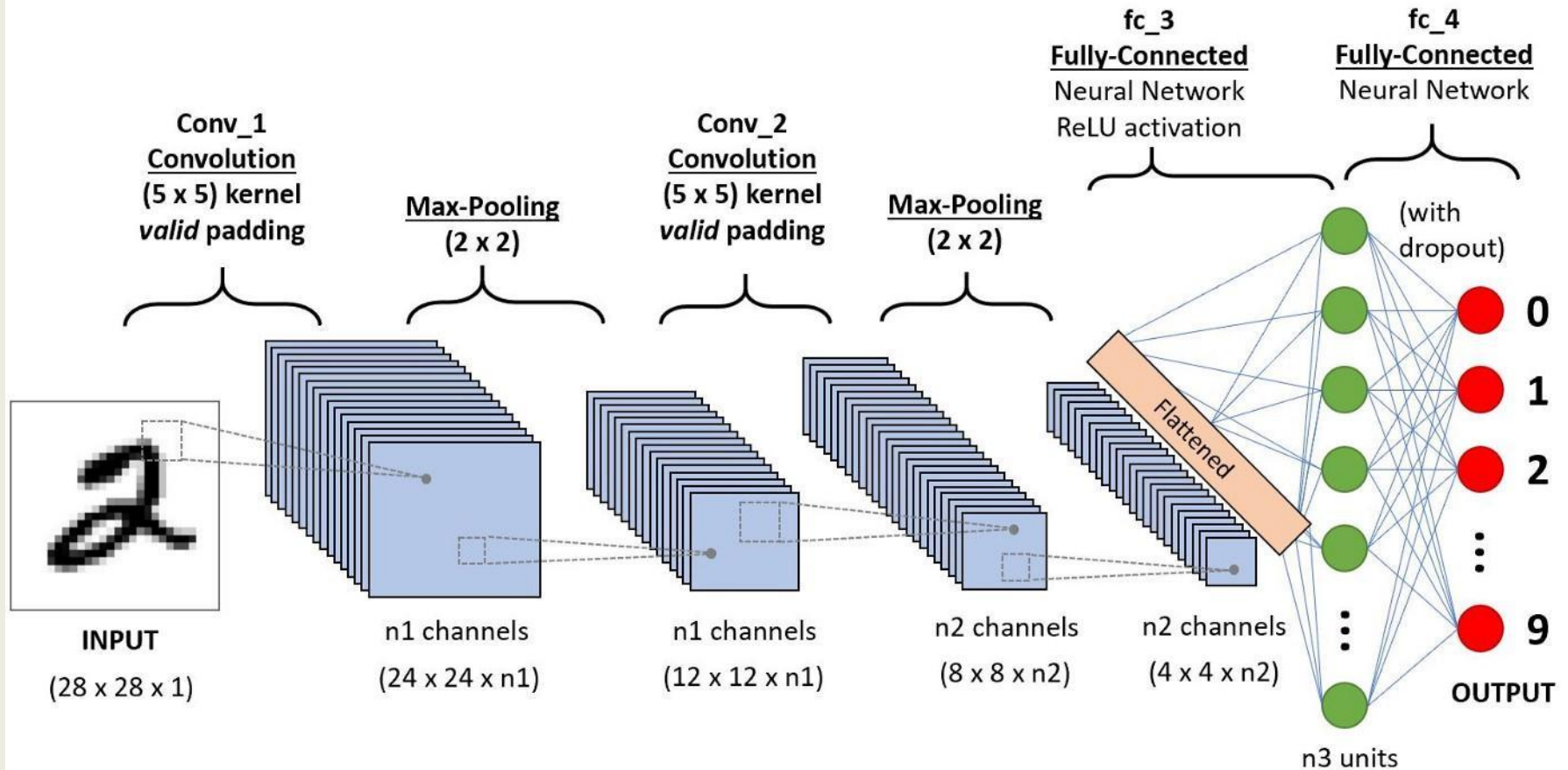
- Do we want the kernel weights to be different for different portions of the image?
- **Yes – It can be**
- This would make the job of learning easier (instead of trying to learn the same weights/kernels at different locations again and again)



Weight Sharing

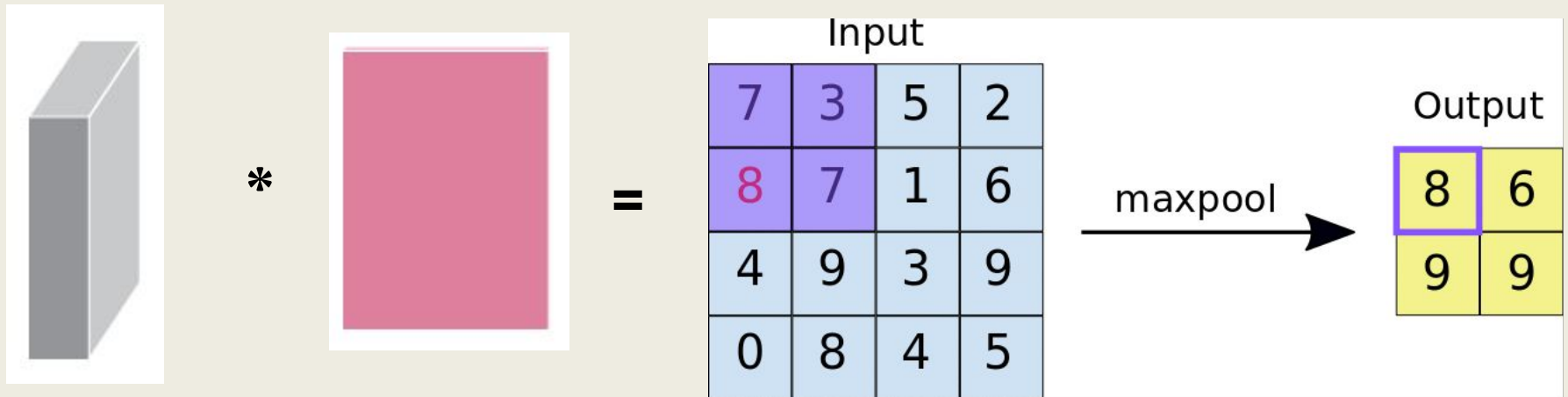


Convolutional Neural Network



Convolutional Neural Network

- Pooling Layer



Convolutional Neural Network

Feature Map

6	6	6	6
4	5	5	4
2	4	4	2
2	4	4	2

Max
Pooling

Average
Pooling

Sum
Pooling

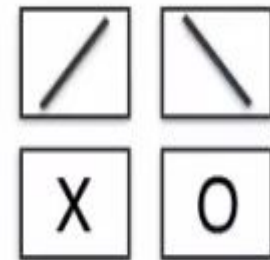
Slightly Complex World



Person



Computer



Alphabet

1	-1	-1
-1	1	-1
-1	-1	1

\

1	-1	1
-1	1	-1
1	-1	1

X

-1	-1	1
-1	1	-1
1	-1	-1

/

-1	1	-1
1	-1	1
-1	1	-1

O

Kernel/ Filter

Feature Map

1	-1
-1	1



-1	1
1	-1



1	-1	1
-1	1	-1
1	-1	1



-1	1	-1
1	-1	1
-1	1	-1

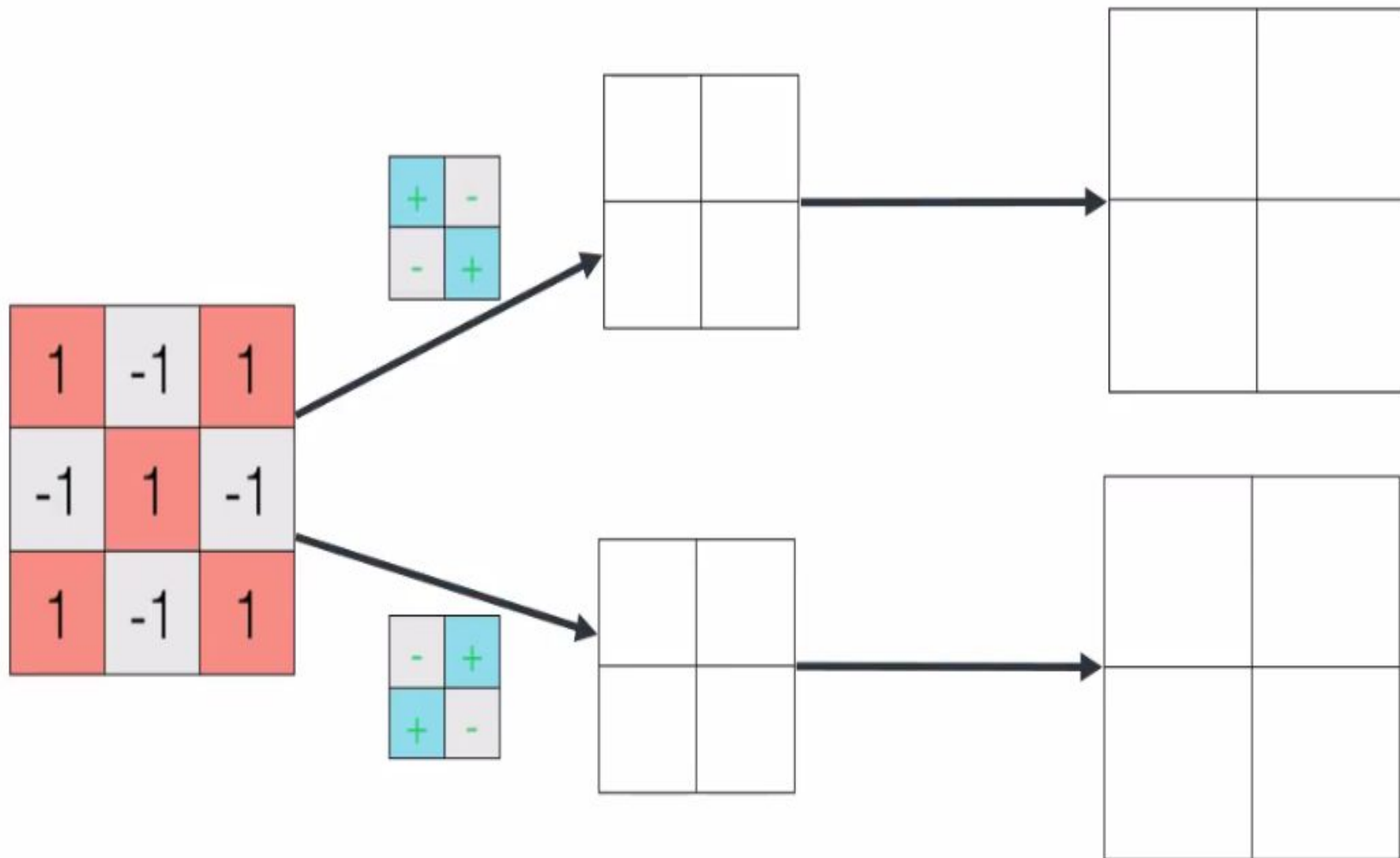


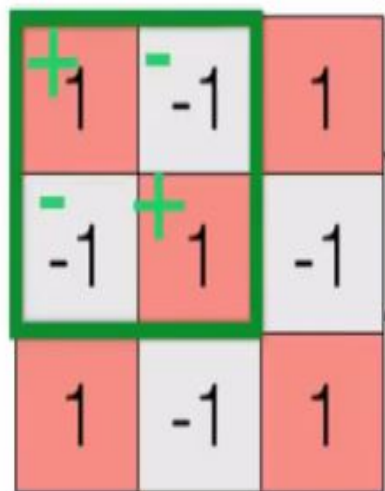
-1	-1	1
-1	1	-1
1	-1	-1

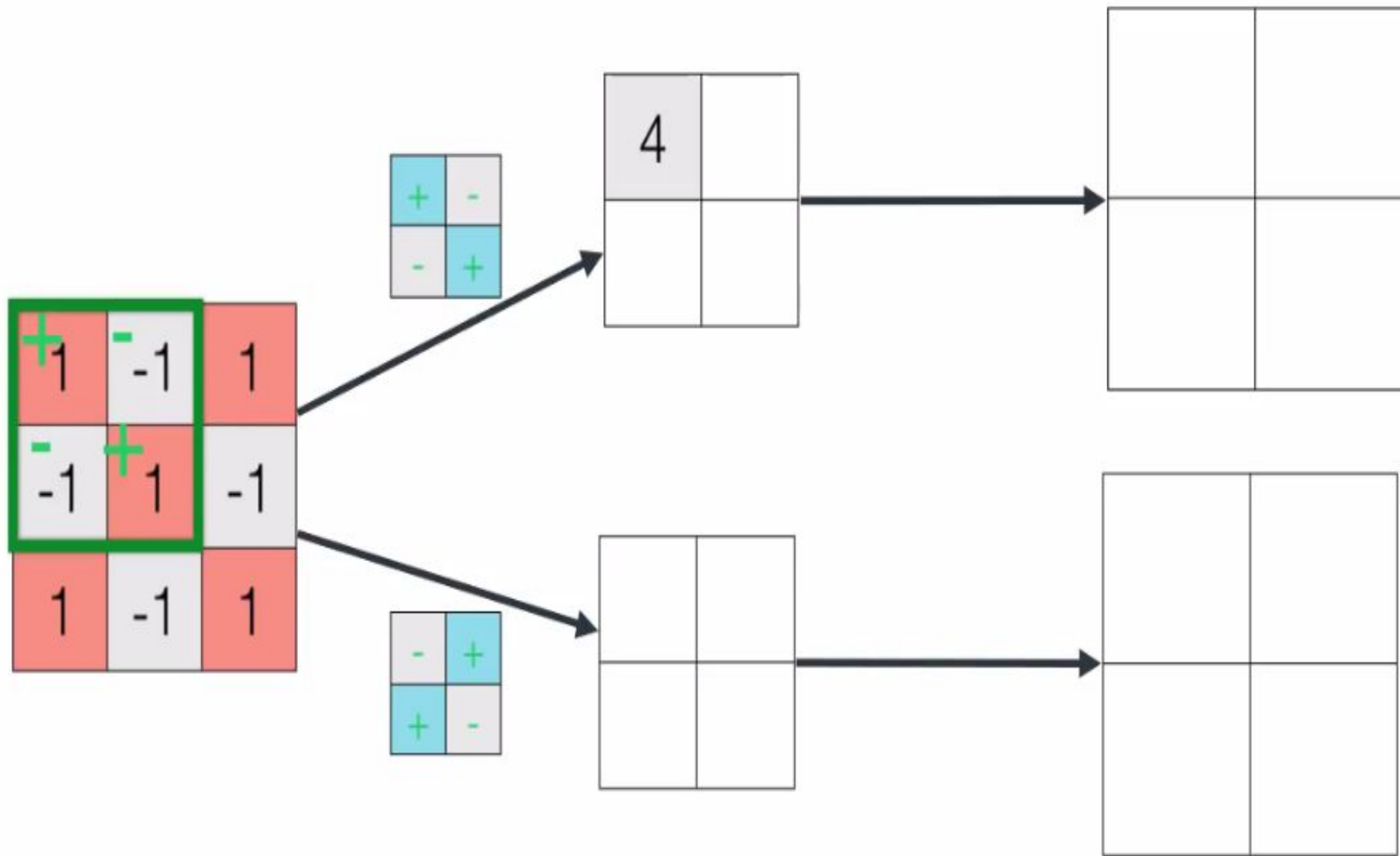


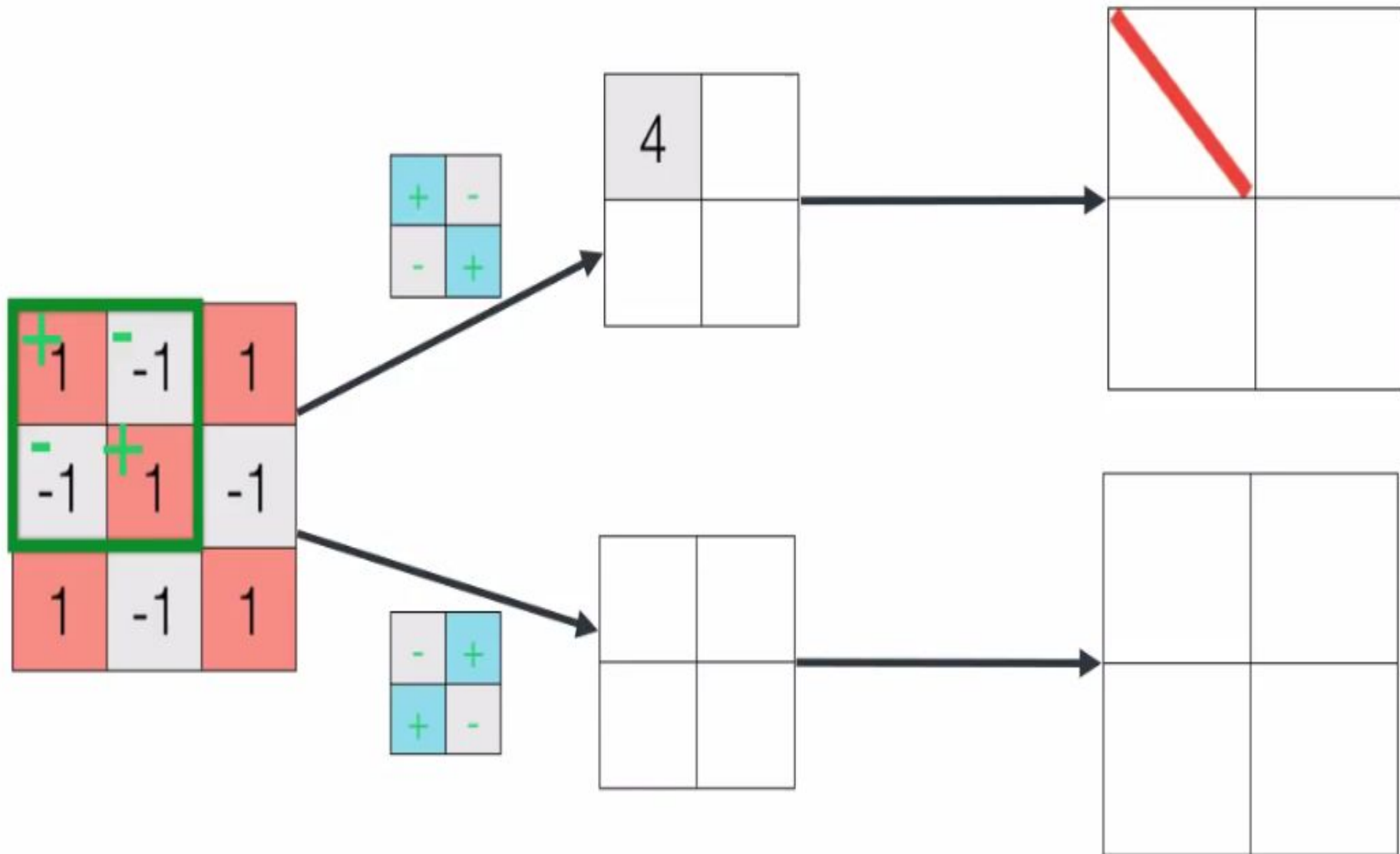
1	-1	-1
-1	1	-1
-1	-1	1

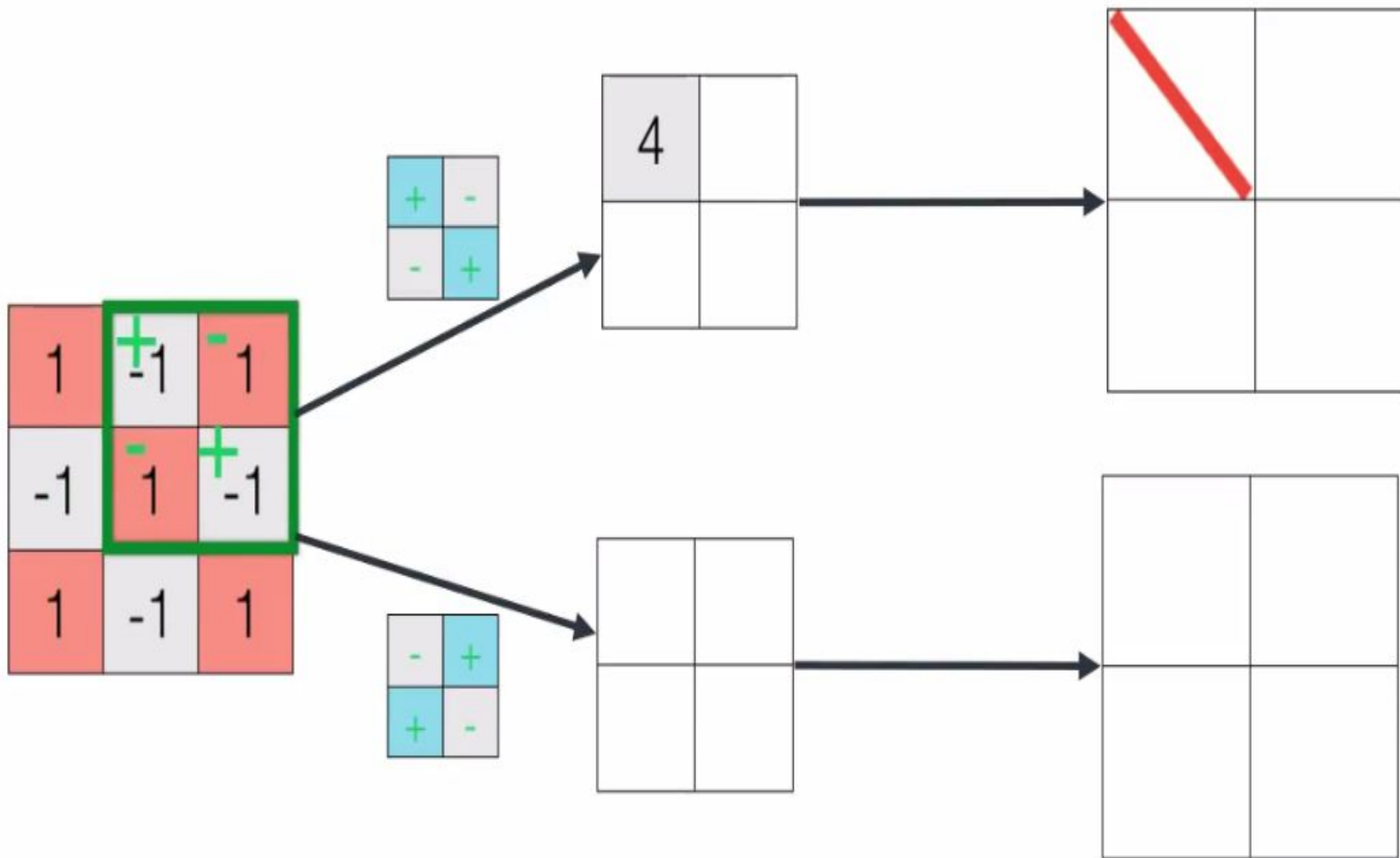


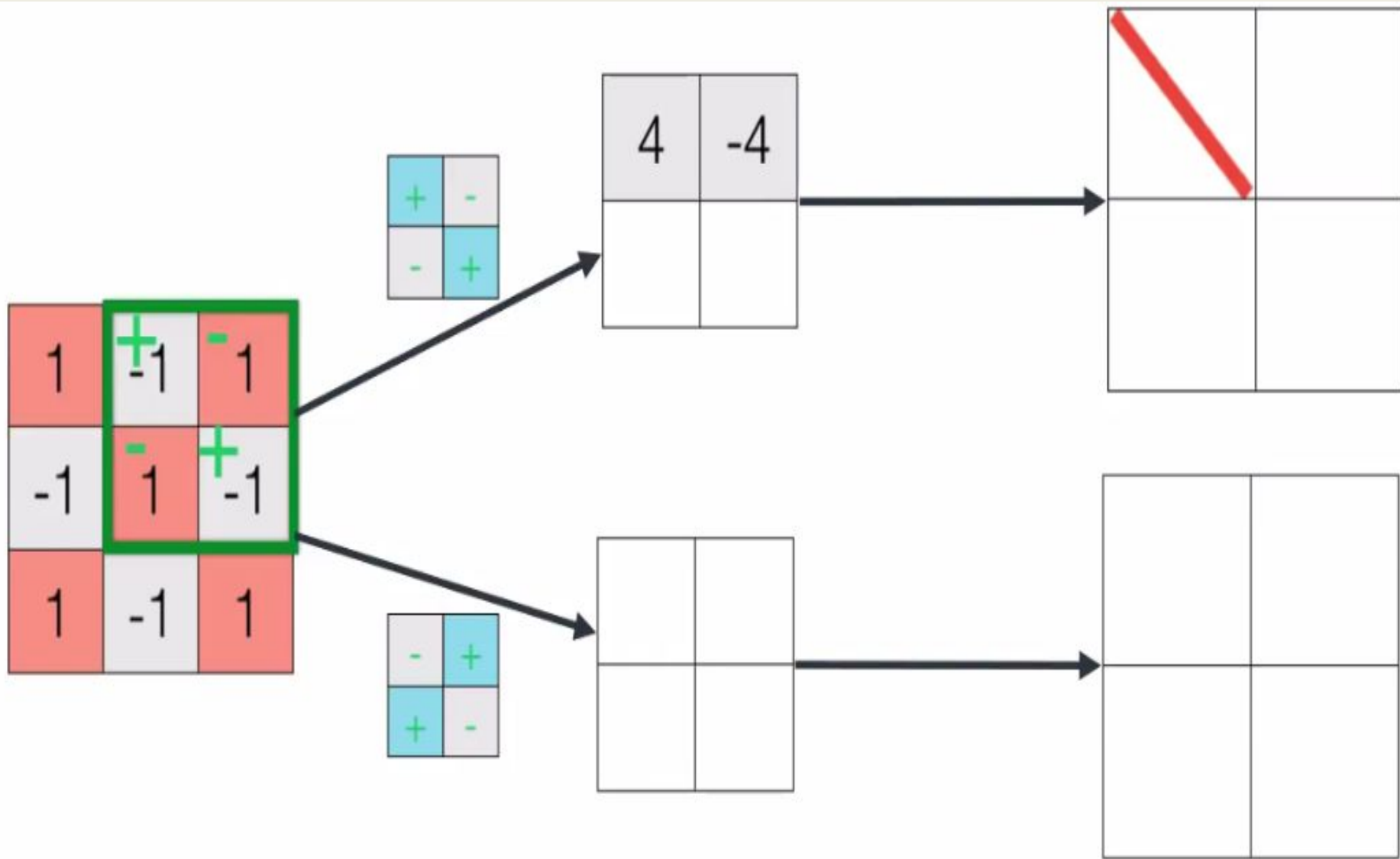


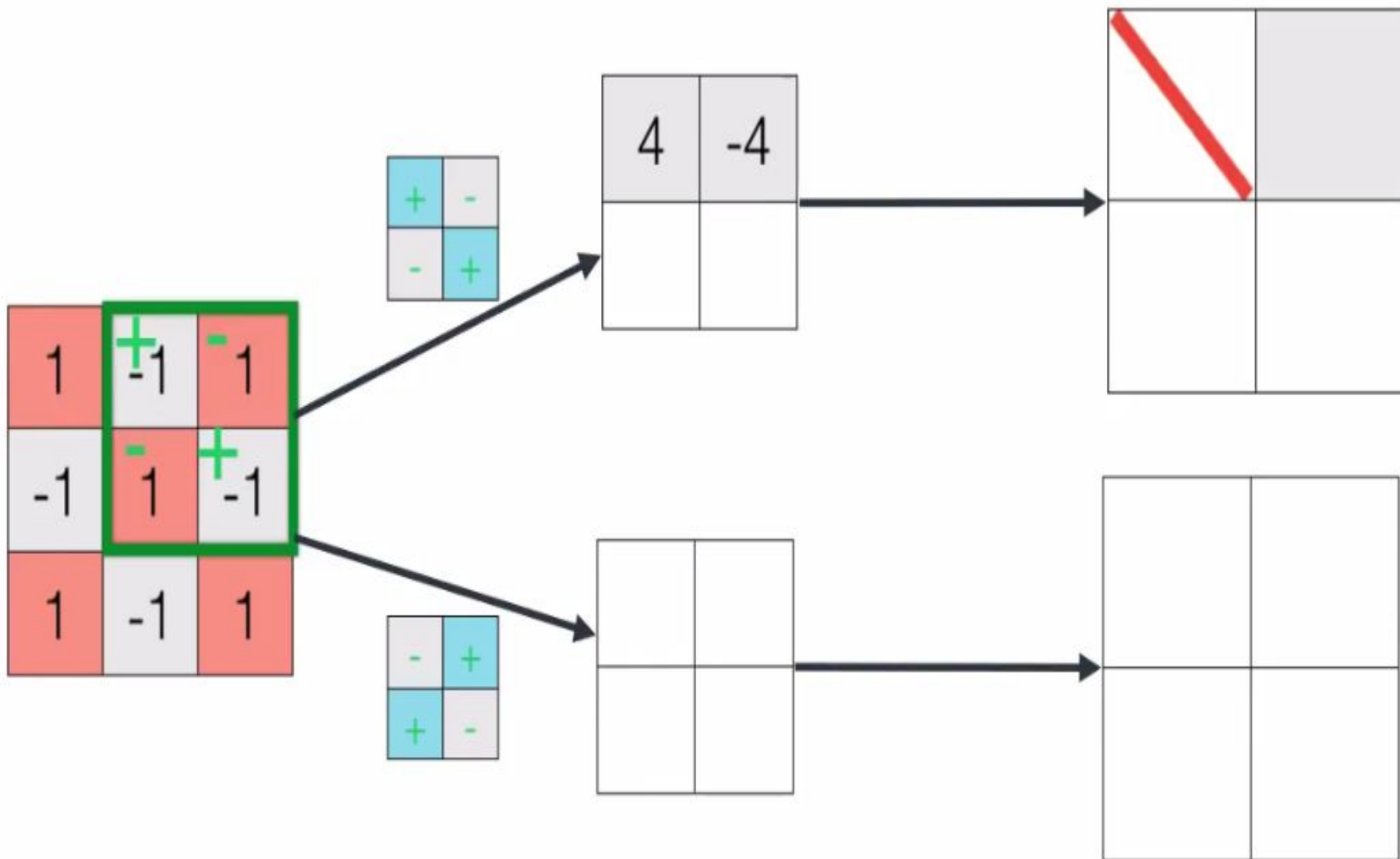


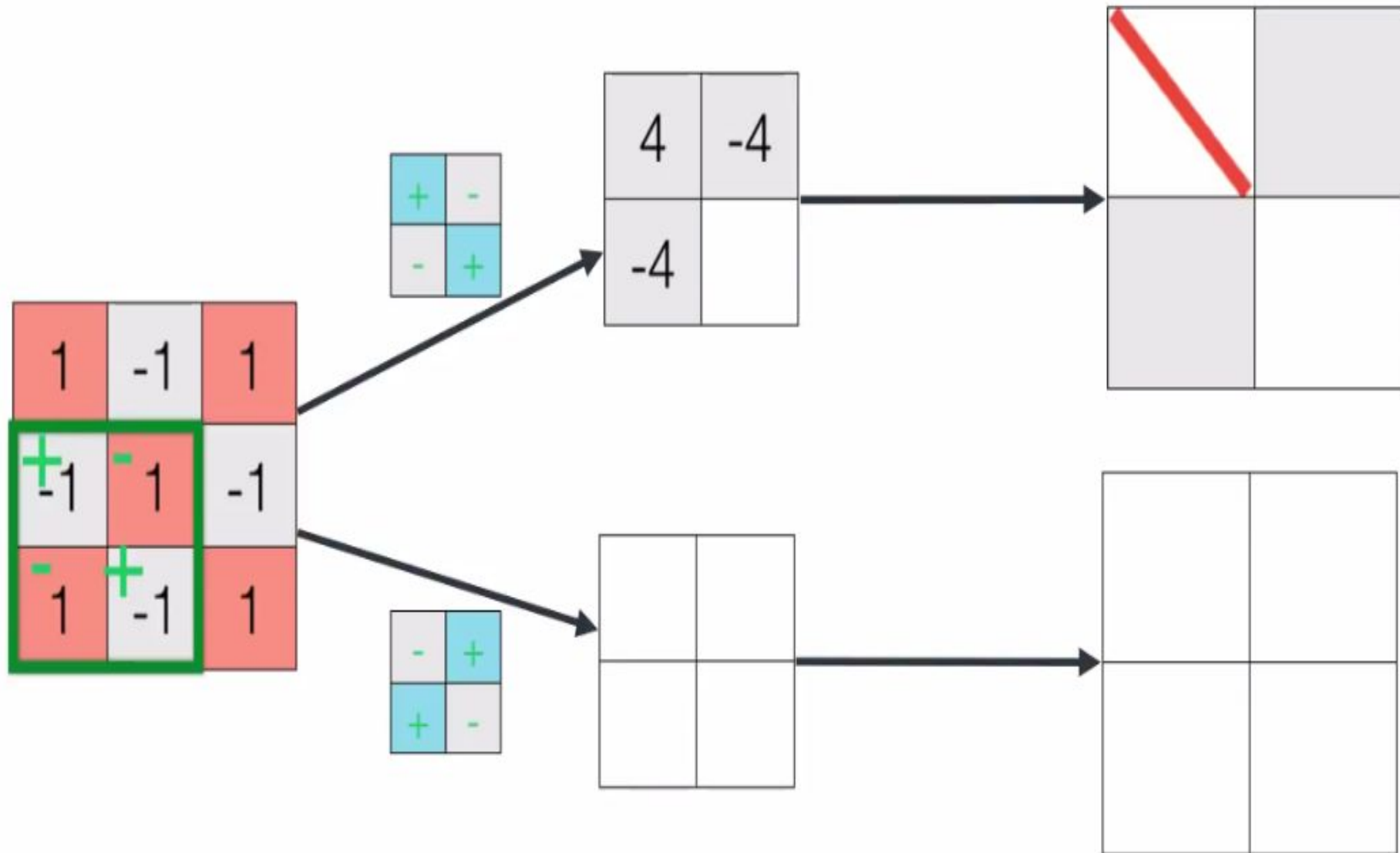


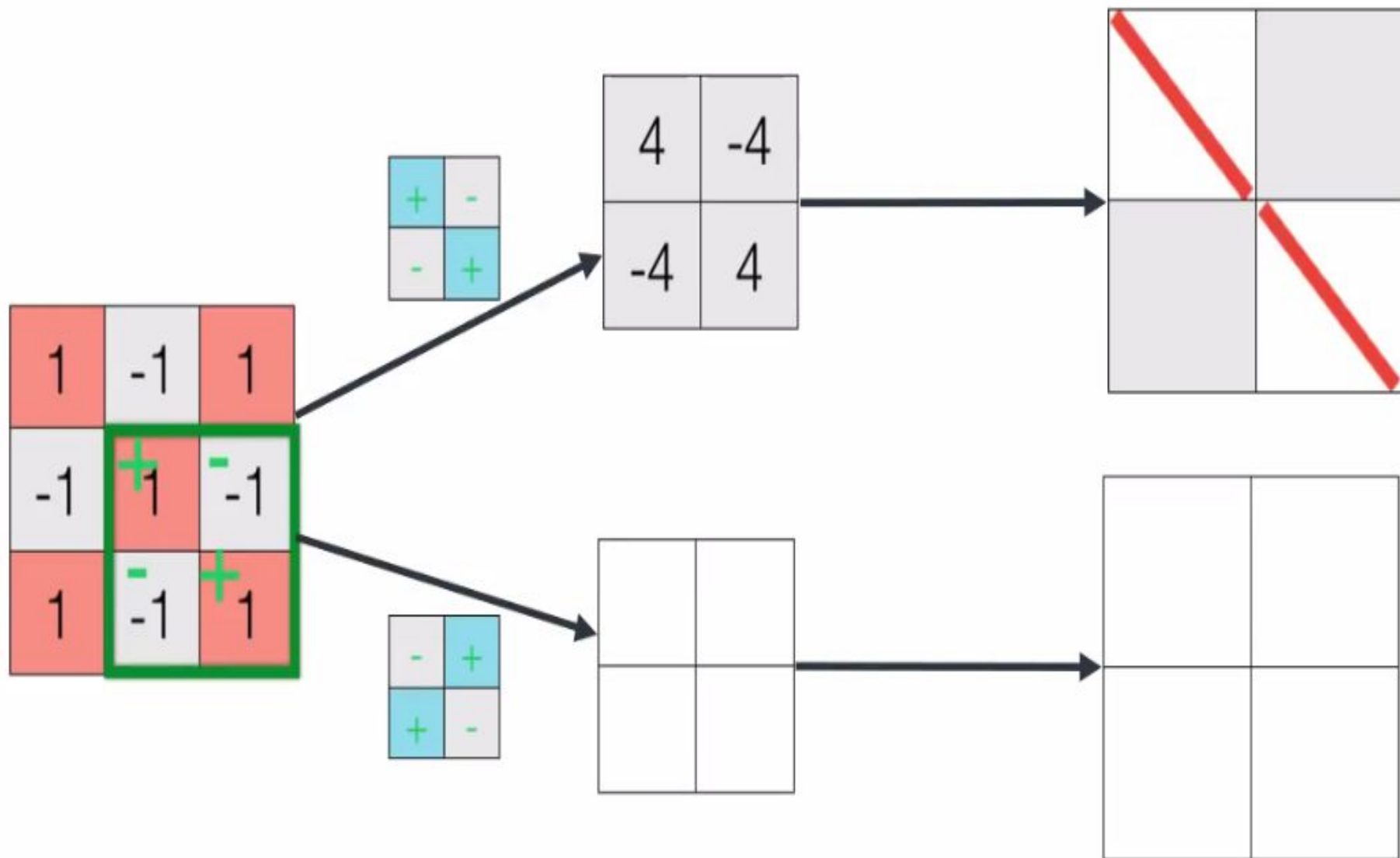


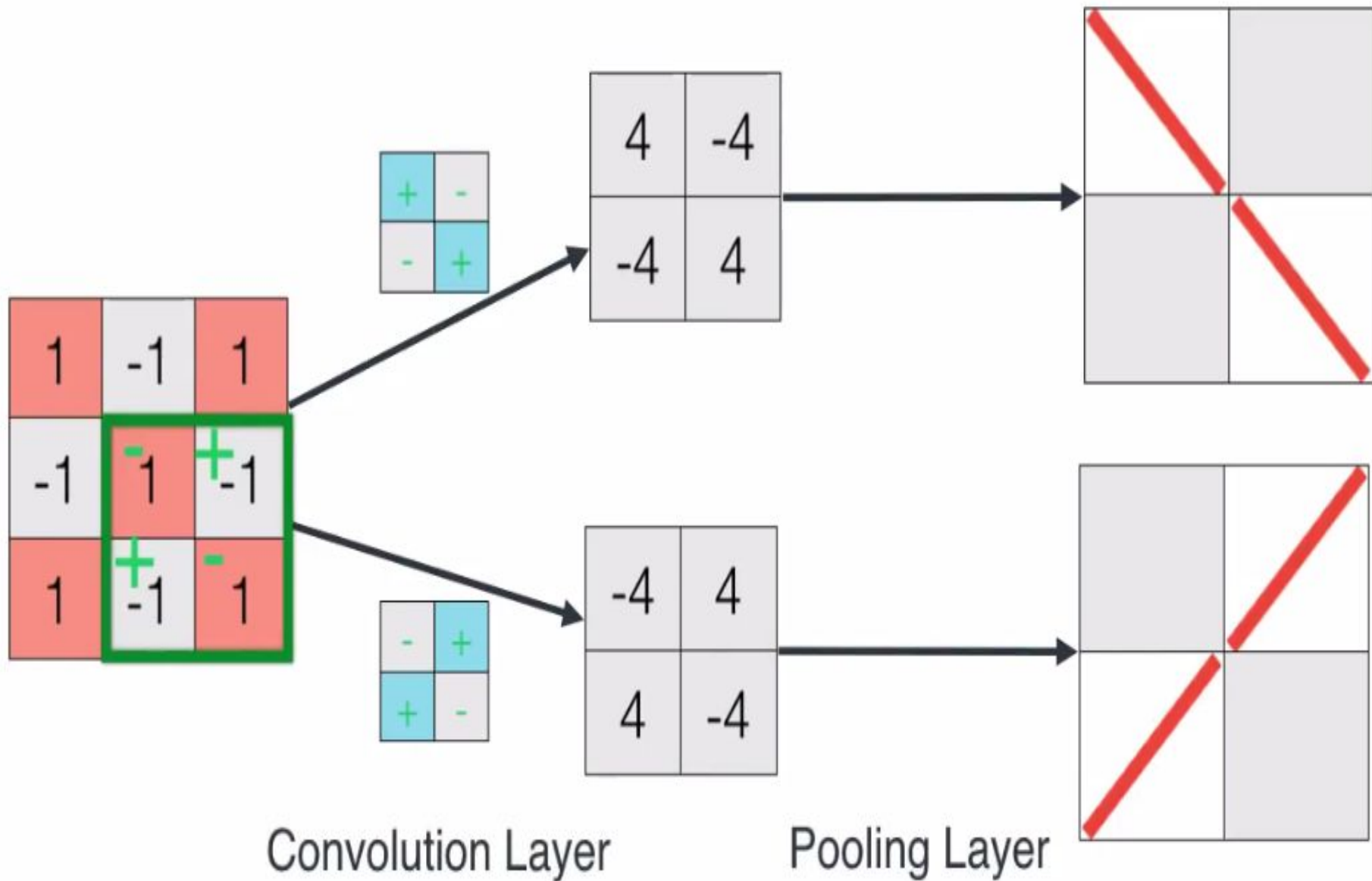




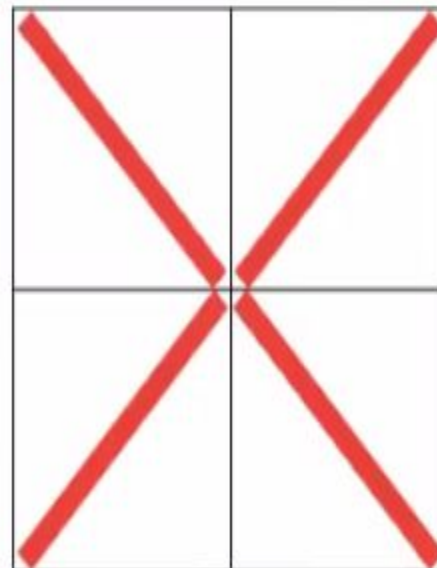


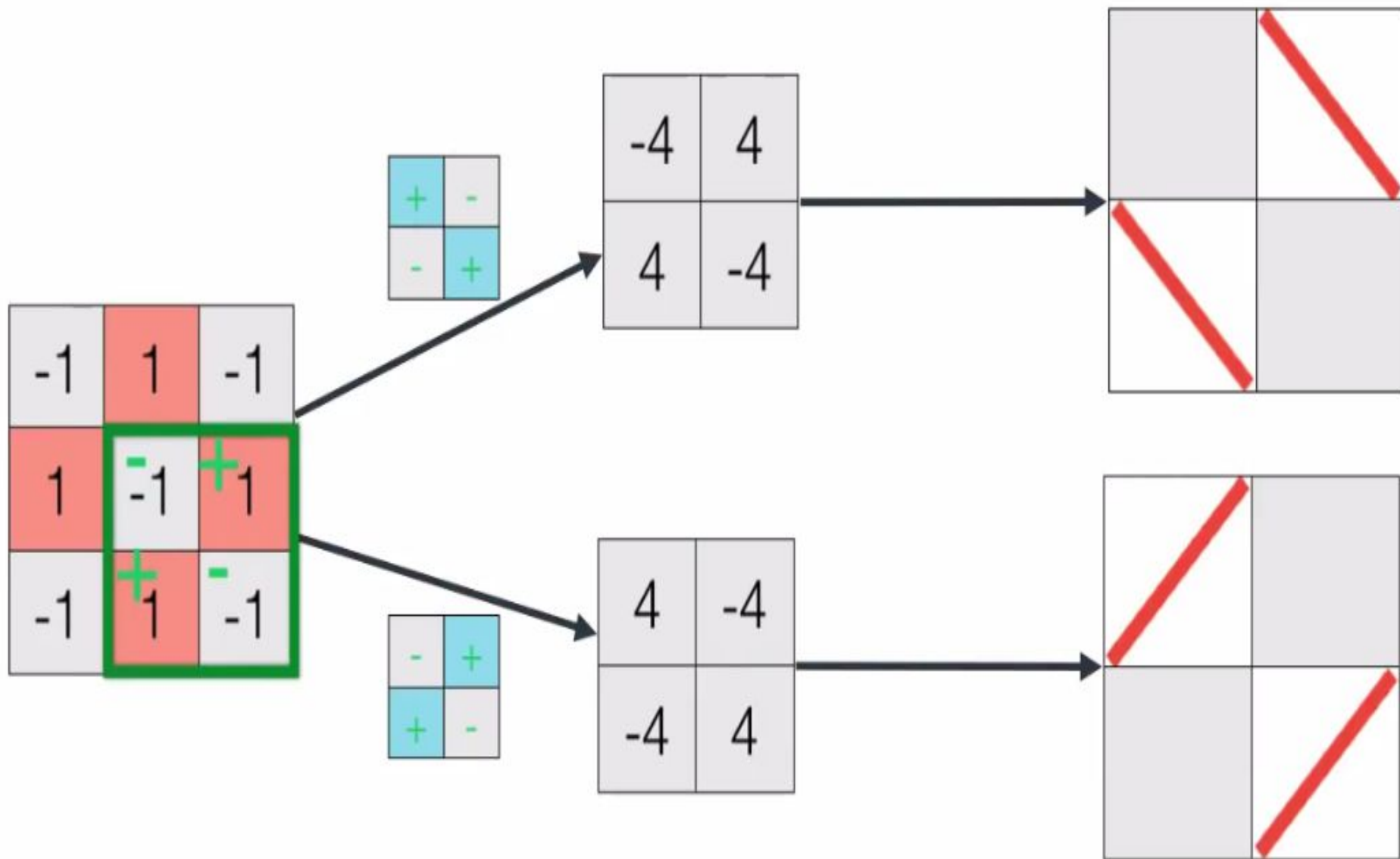


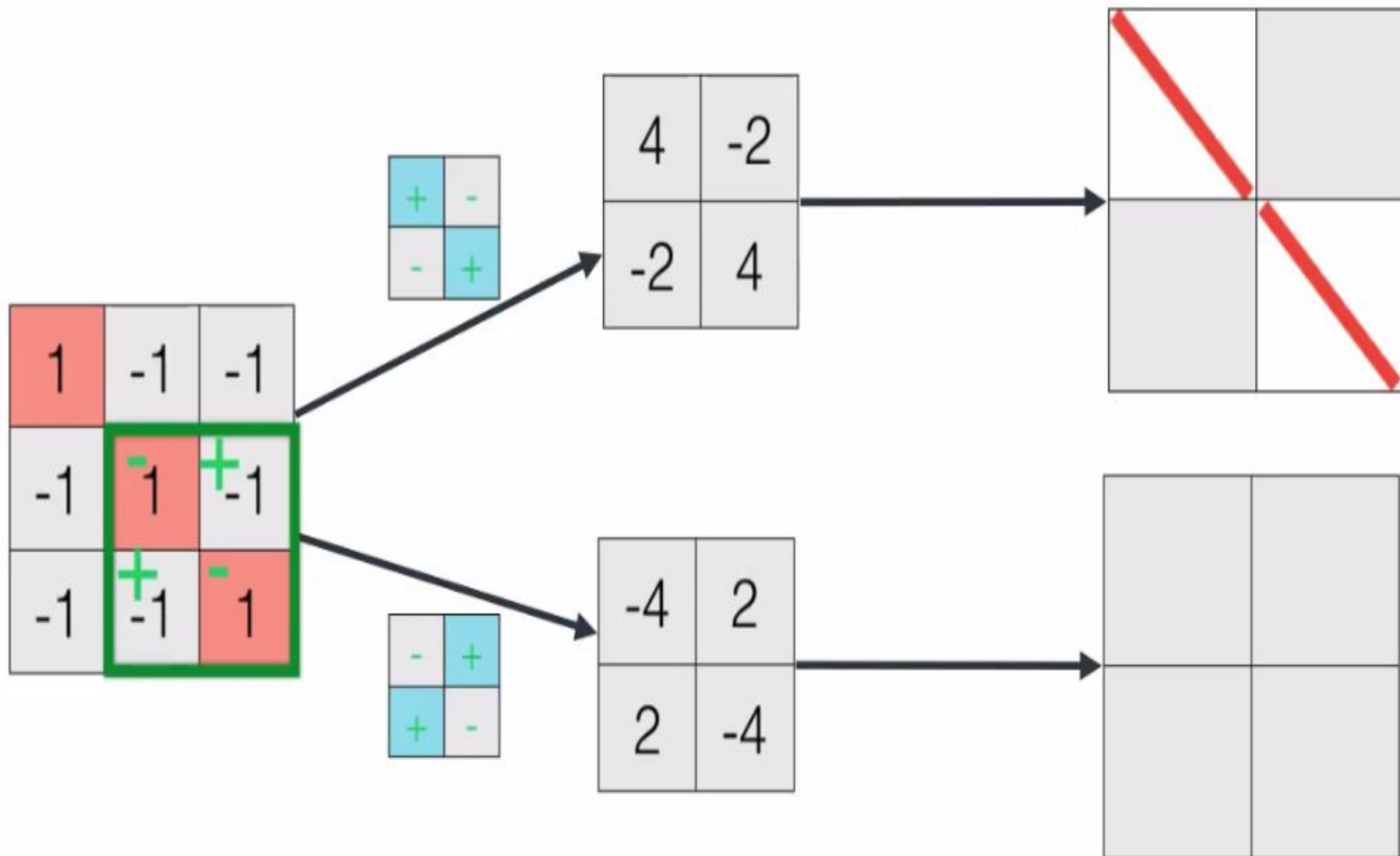




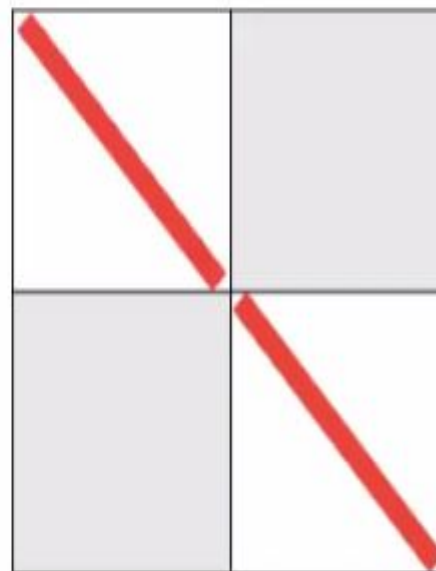
1	-1	1
-1	1	-1
1	-1	1

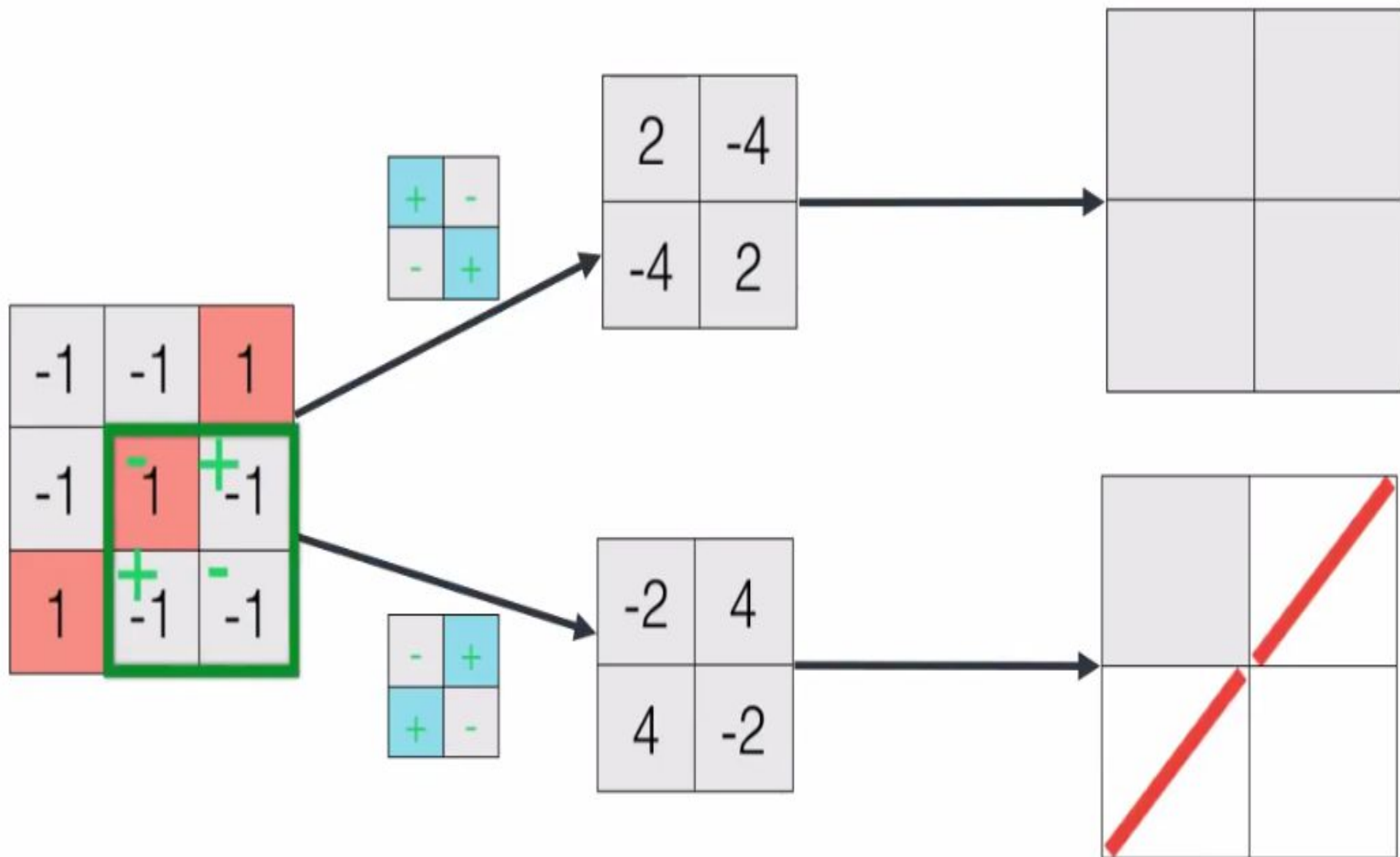




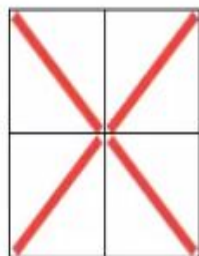


1	-1	-1
-1	1	-1
-1	-1	1





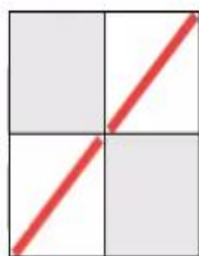
1	-1	1
-1	1	-1
1	-1	1



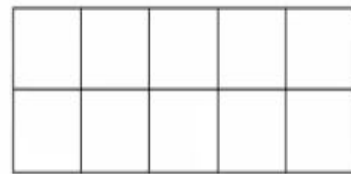
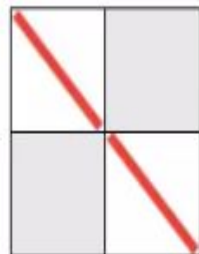
-1	1	-1
1	-1	1
-1	1	-1



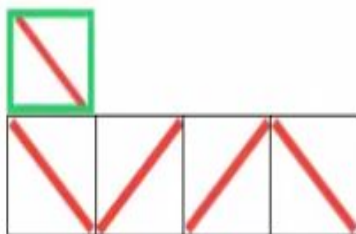
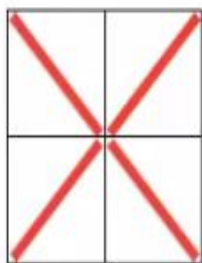
-1	-1	1
-1	1	-1
1	-1	-1



1	-1	-1
-1	1	-1
-1	-1	1

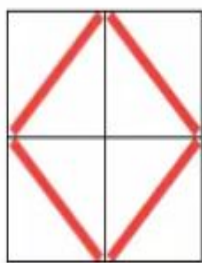


1	-1	1
-1	1	-1
1	-1	1

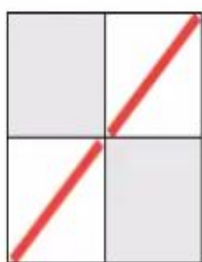


	1			

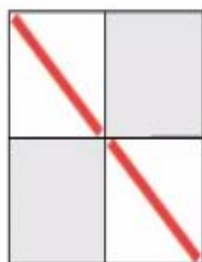
-1	1	-1
1	-1	1
-1	1	-1



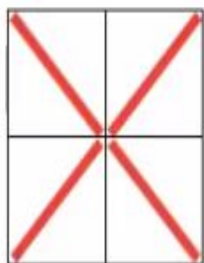
-1	-1	1
-1	1	-1
1	-1	-1



1	-1	-1
-1	1	-1
-1	-1	1

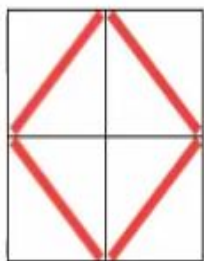


1	-1	1
-1	1	-1
1	-1	1

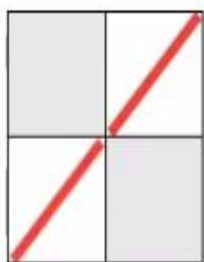


	1	-1	-1	1

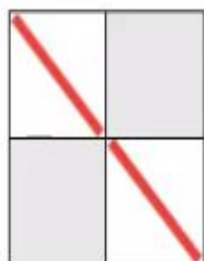
-1	1	-1
1	-1	1
-1	1	-1



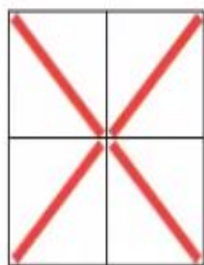
-1	-1	1
-1	1	-1
1	-1	-1



1	-1	-1
-1	1	-1
-1	-1	1

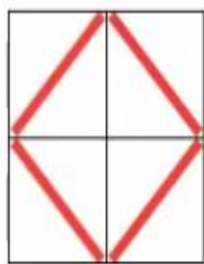


1	-1	1
-1	1	-1
1	-1	1

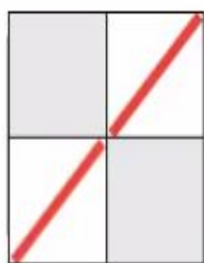


	1	-1	-1	1

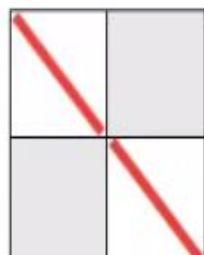
-1	1	-1
1	-1	1
-1	1	-1



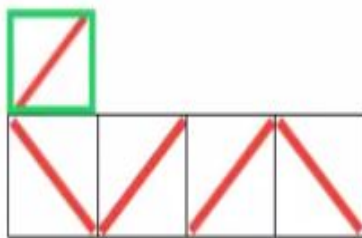
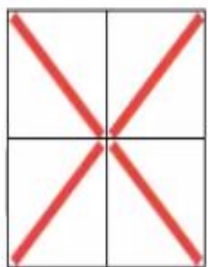
-1	-1	1
-1	1	-1
1	-1	-1



1	-1	-1
-1	1	-1
-1	-1	1

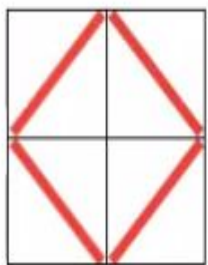


1	-1	1
-1	1	-1
1	-1	1

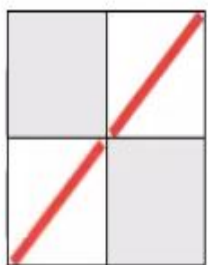


	1	-1	-1	1
	-1			

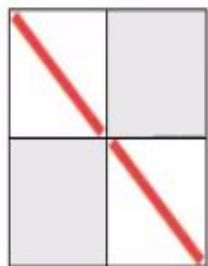
-1	1	-1
1	-1	1
-1	1	-1



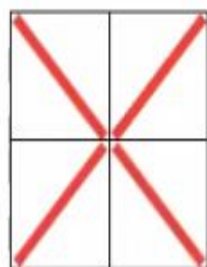
-1	-1	1
-1	1	-1
1	-1	-1



1	-1	-1
-1	1	-1
-1	-1	1



1	-1	1
-1	1	-1
1	-1	1

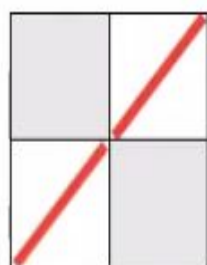


	1	-1	-1	1
	-1	1	1	-1

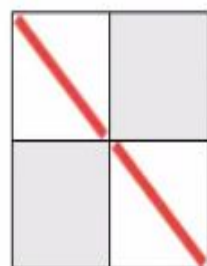
-1	1	-1
1	-1	1
-1	1	-1



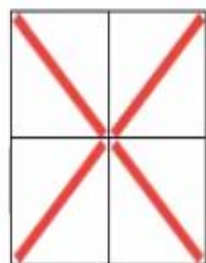
-1	-1	1
-1	1	-1
1	-1	-1



1	-1	-1
-1	1	-1
-1	-1	1



1	-1	1
-1	1	-1
1	-1	1



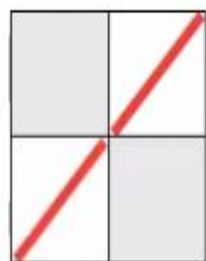
	1	-1	-1	1
	-1	1	1	-1

-1	1	-1
1	-1	1
-1	1	-1



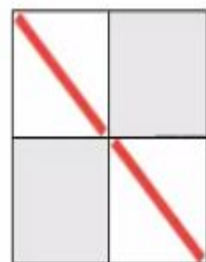
	-1	1	1	-1
	1	-1	-1	1

-1	-1	1
-1	1	-1
1	-1	-1



	-1	-1	-1	-1
	-1	1	1	-1

1	-1	-1
-1	1	-1
-1	-1	1



	1	-1	-1	1
	-1	-1	-1	-1

1	-1	1
-1	1	-1
1	-1	1



	1	-1	-1	1
	-1	1	1	-1

-1	1	-1
1	-1	1
-1	1	-1



	-1	1	1	-1
	1	-1	-1	1

-1	-1	1
-1	1	-1
1	-1	-1



	-1	-1	-1	-1
	-1	1	1	-1

1	-1	-1
-1	1	-1
-1	-1	1



	1	-1	-1	1
	-1	-1	-1	-1

Filters

+	-	-	+
-	+	+	-



-	+	+	-
+	-	-	+



-	-	-	-
-	+	+	-



+	-	-	+
-	-	-	-



1	-1	1
-1	1	-1
1	-1	1



	1	-1	-1	1
	-1	1	1	-1

+	-	-	+
-	+	+	-

X

-	+	+	-
+	-	-	+

O

-	-	-	-
-	+	+	-

/

+	-	-	+
-	-	-	-

\

1	-1	1
-1	1	-1
1	-1	1

+1



1	1	-1	-1	1
-1	-1	1	1	-1

+	-	-	+
-	+	+	-

X

-	+	+	-
+	-	-	+

O

-	-	-	-
-	+	+	-

/

+	-	-	+
-	-	-	-

\

1	-1	1
-1	1	-1
1	-1	1



	1	-1	-1	1
	-1	1	1	-1

+	-	-	+
-	+	+	-



-	+	+	-
+	-	-	+



-	-	-	-
-	+	+	-



+	-	-	+
-	-	-	-



+1 +1 +1 +1
 +1 +1 +1 +1

1	-1	1
-1	1	-1
1	-1	1



	1	-1	-1	1
	-1	1	1	-1

+	-	-	+
-	+	+	-

X

8

-	+	+	-
+	-	-	+

O

-	-	-	-
-	+	+	-

/

+	-	-	+
-	-	-	-

\

+1 +1 +1 +1
+1 +1 +1 +1

1	-1	1
-1	1	-1
1	-1	1



	1	-1	-1	1
	-1	1	1	-1

+	-	-	+
-	+	+	-



8

-	+	+	-
+	-	-	+



-8

-	-	-	-
-	+	+	-



4

+	-	-	+
-	-	-	-



4

1	-1	1
-1	1	-1
1	-1	1



	1	-1	-1	1
	-1	1	1	-1

+	-	-	+
-	+	+	-



8



-	+	+	-
+	-	-	+



-8

-	-	-	-
-	+	+	-



4

+	-	-	+
-	-	-	-



4

-1	1	-1
1	-1	1
-1	1	-1



	-1	1	1	-1
	1	-1	-1	1

+	-	-	+
-	+	+	-



-8

-	+	+	-
+	-	-	+



8



-	-	-	-
-	+	+	-



-4

+	-	-	+
-	-	-	-



-4

-1	-1	1
-1	1	-1
1	-1	-1



	-1	-1	-1	-1
	-1	1	1	-1

+	-	-	+
-	+	+	-



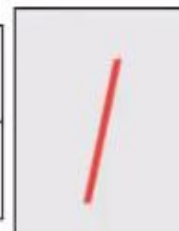
4

-	+	+	-
+	-	-	+



-4

-	-	-	-
-	+	+	-



8



+	-	-	+
-	-	-	-



0

1	-1	-1
-1	1	-1
-1	-1	1



	1	-1	-1	1
	-1	-1	-1	-1

+	-	-	+
-	+	+	-

X

4

-	+	+	-
+	-	-	+

O

-4

-	-	-	-
-	+	+	-

/

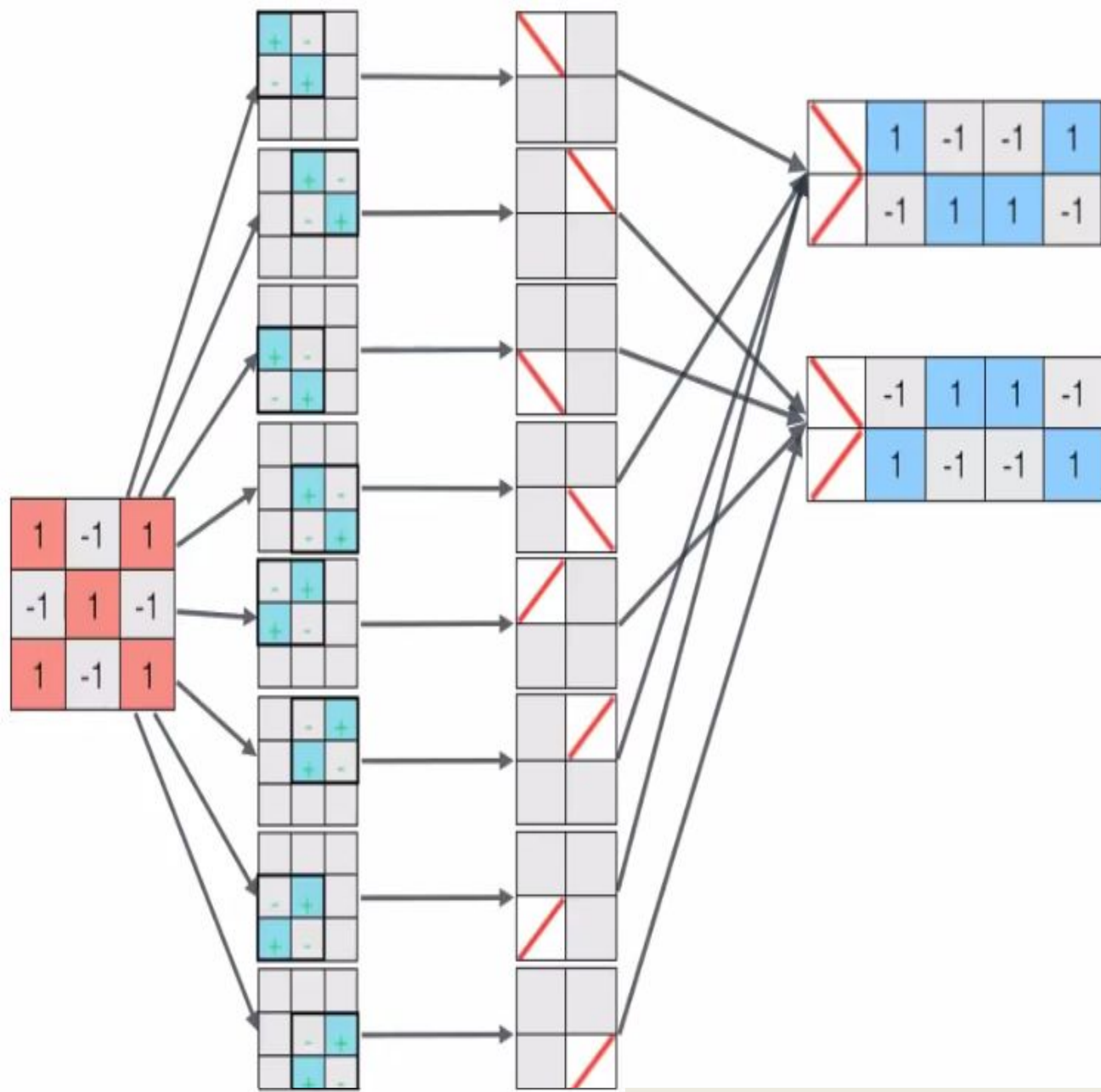
0

+	-	-	+
-	-	-	-

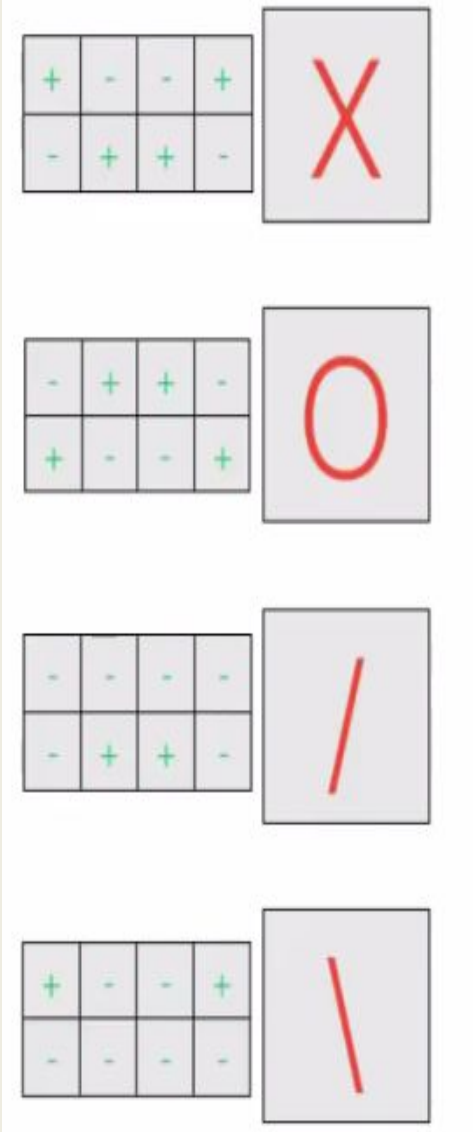
\

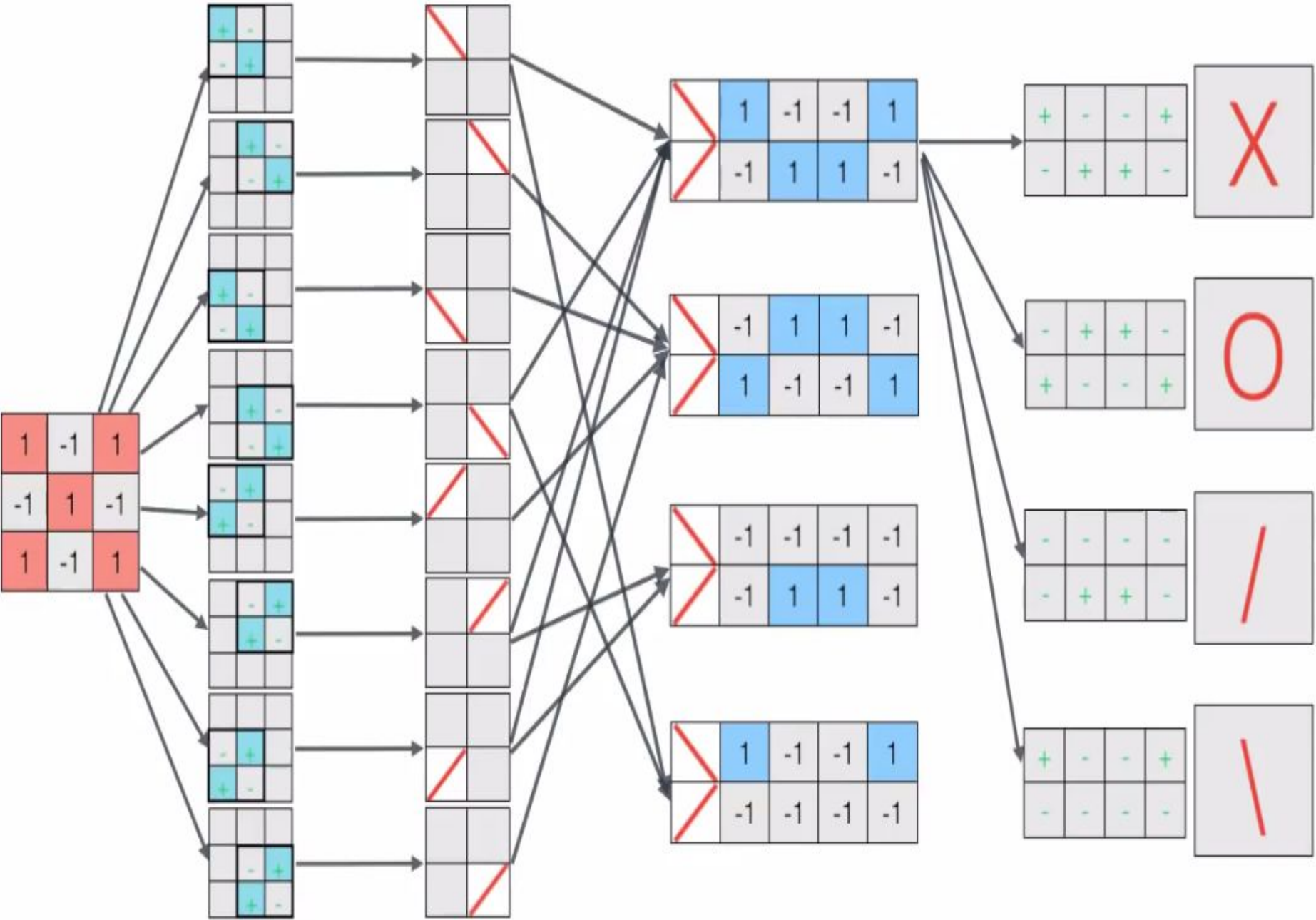
8

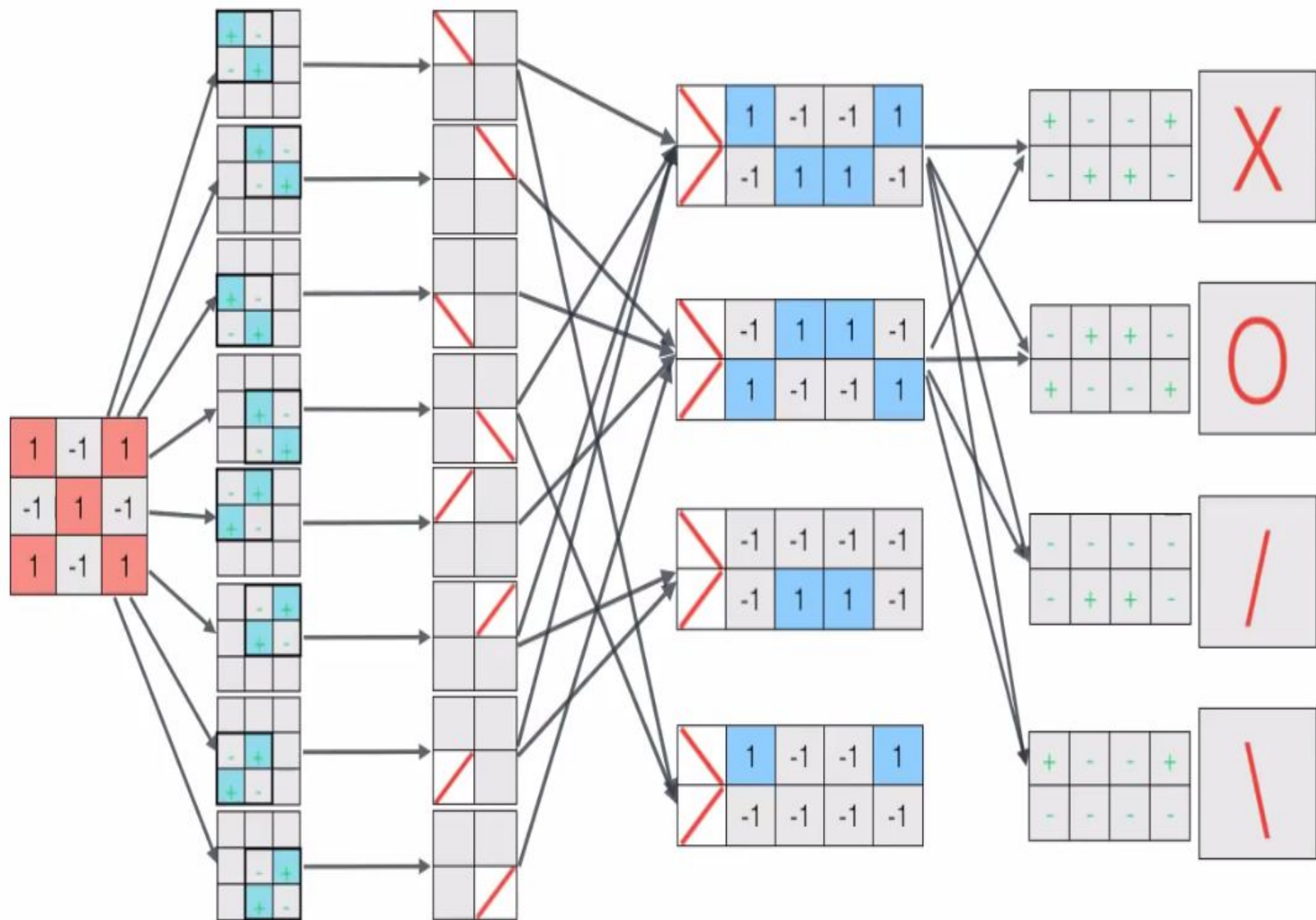


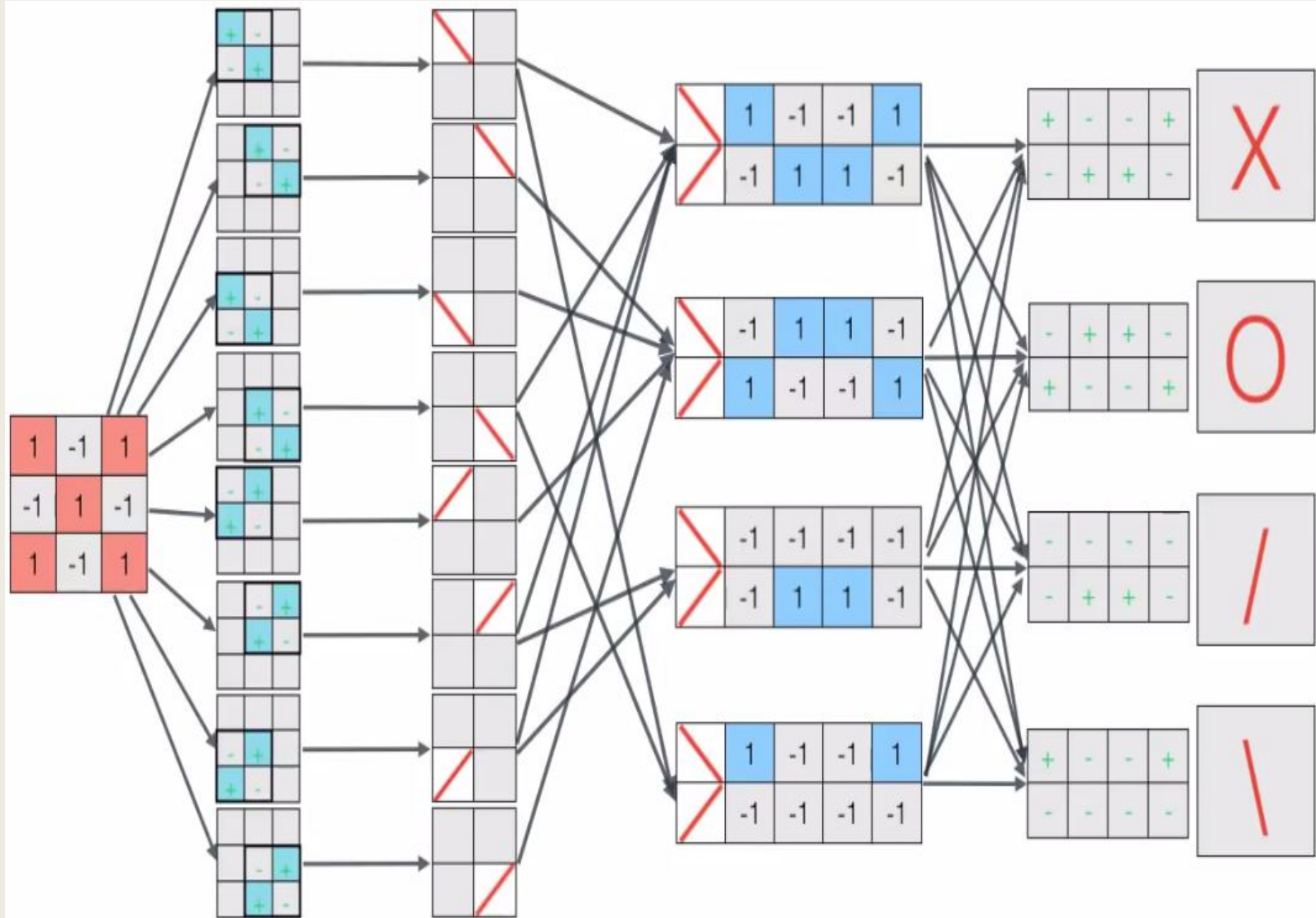


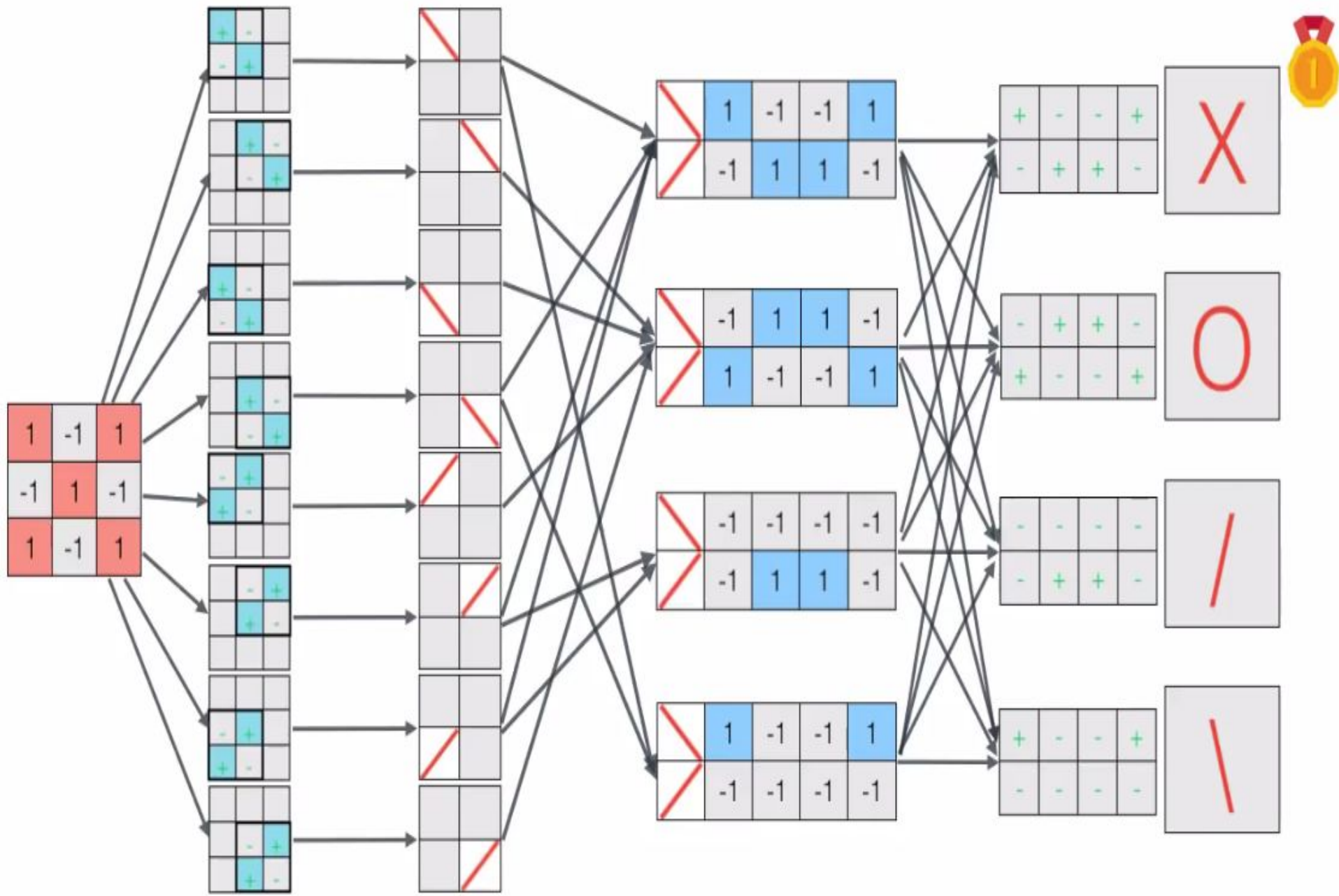
Convolution Layer Pooling Layer





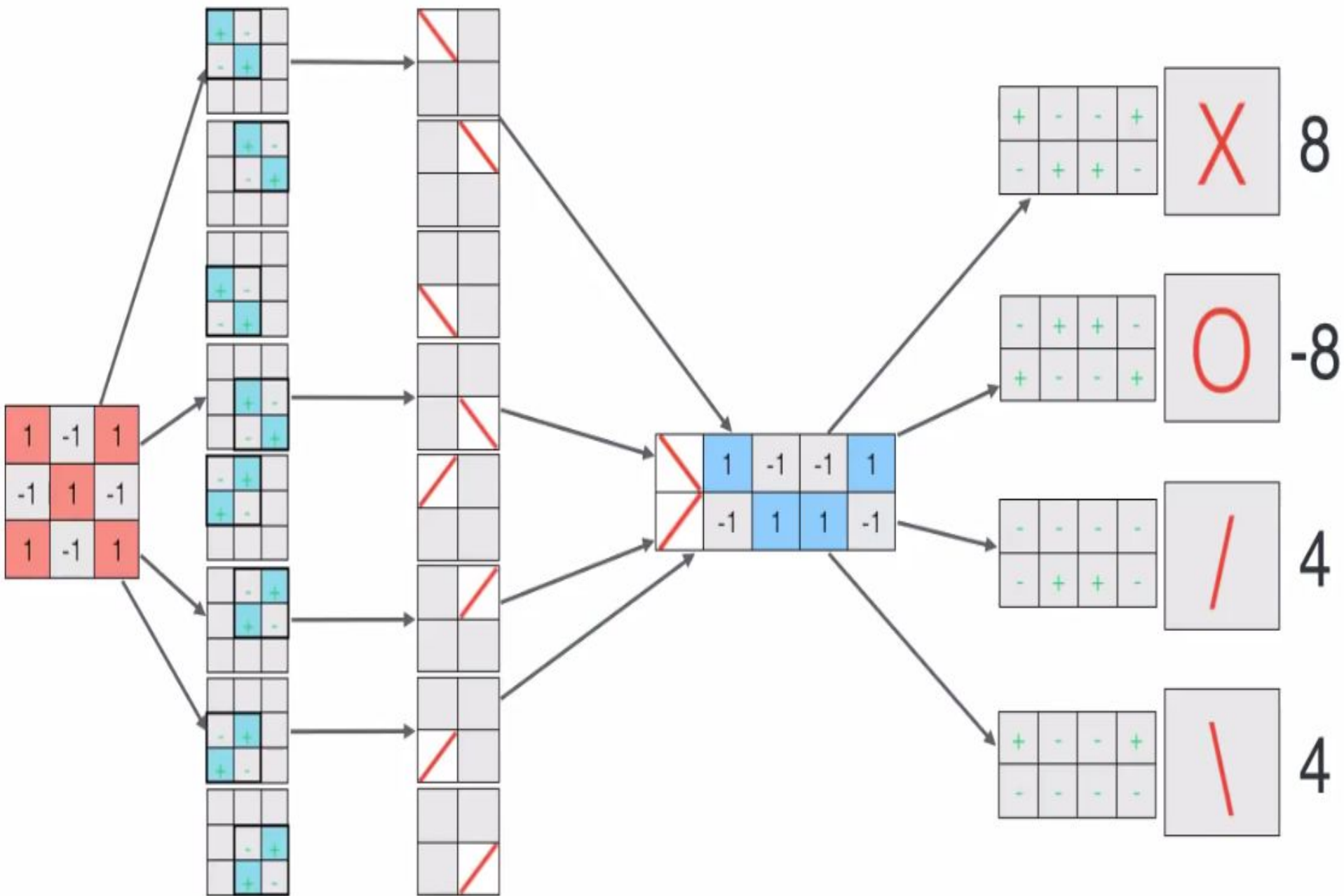






Convolution Layer Pooling Layer

Fully Connected Layer



Convolution Layer Pooling Layer

Fully Connected Layer

Recap

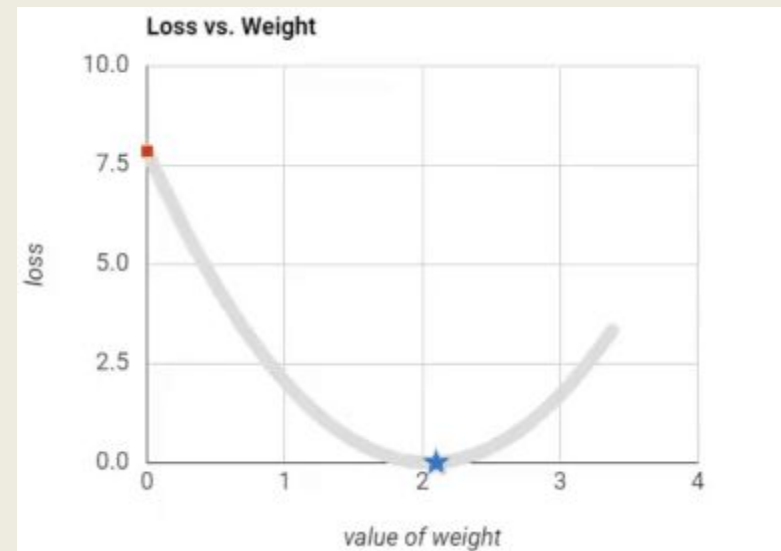
- Convolutional Neural Network
- Convolution Operation
- Filter/ Kernels
 - Feature Maps
- Relationship between I/P, O/P, and filters
 - Padding, Stride, Depth
- Connection between CNN and FNN (NN)
 - Sparsity
 - Weight Sharing
 - Regularizes
 - Optimizers
- Activation functions
 - ReLU with smaller initialization
 - Leaky ReLU

Optimizers

- SGD – Stochastic Gradient Descent
- RMSprop
- Adam
- Adadelata
- Adagrad
- Adamax
- Nadam
- Ftrl

Stochastic Gradient Descent

- **Gradient descent** - a *first-order iterative optimization algorithm* for finding the minimum of a function
- The goal of the gradient descent is to minimize a given function - the *loss function of the neural network*.
- Compute the slope (gradient) that is the first-order derivative of the function at the current point
- Move-in the opposite direction of the slope increase from the current point by the computed amount



Types of Gradient Descent

- **Stochastic Gradient Descent (SGD)** - variation of the gradient descent algorithm that *calculates the error and updates the model for each example* in the training dataset.
- SGD – online machine learning model - data becomes available in a sequential order and is used to update the best predictor for future data at each step

Stochastic Gradient Descent (SGD)

- Pros
 - The frequent updates immediately give an insight into the performance of the model and the *rate of improvement*.
 - Simplest to understand and implement, especially for beginners.
 - The increased model update frequency can *result in faster learning* on some problems.
 - The noisy update process can allow the model to avoid local minima (e.g. premature convergence).
- Cons
 - Updating the model so frequently is more *computationally expensive*, taking significantly longer to train models on large datasets.
 - The frequent updates can result in a noisy gradient signal

Batch Gradient Descent (BGD)

- **Batch gradient descent** calculates the error for each example in the training dataset, but only *updates the model after all training examples* have been evaluated.
- One cycle through the entire training dataset is called a training *epoch*.
- It is often said that batch gradient descent performs model updates at the end of each training epoch.

Batch Gradient Descent (BGD)

- **Pros**

- Fewer updates to the model means it is more *computationally efficient* than stochastic gradient descent.
- Results in a *more stable error gradient* and may result in a more *stable convergence* on some problems.

- **Cons**

- The more stable error gradient may result in *premature convergence* of the model to a *less optimal set of parameters*.
- The updates at the *end of the training epoch* require the additional complexity of *accumulating prediction errors* across all training examples.
- Requires the *entire training dataset in memory* and available to the algorithm.
- *Model updates*, and in turn *training speed*, may become very slow for large datasets.

Mini-Batch Gradient Descent

- **Mini-batch gradient descent** splits the *training dataset* into *small batches* that are used to calculate *model error* and *update model coefficients*.
- Implementations may choose to *sum the gradient* over the mini-batch which further reduces the variance of the gradient.
- Mini-batch gradient descent seeks to find a balance between the robustness of stochastic gradient descent and the efficiency of batch gradient descent.

Mini-Batch Gradient Descent

- **Pros**

- The model update frequency is higher than batch gradient descent which allows for a more *robust convergence*, *avoiding local minima*.
- The batched updates provide a *computationally more efficient* process than stochastic gradient descent.
- The batching allows both the efficiency of *not having all training data* in memory and algorithm implementations.

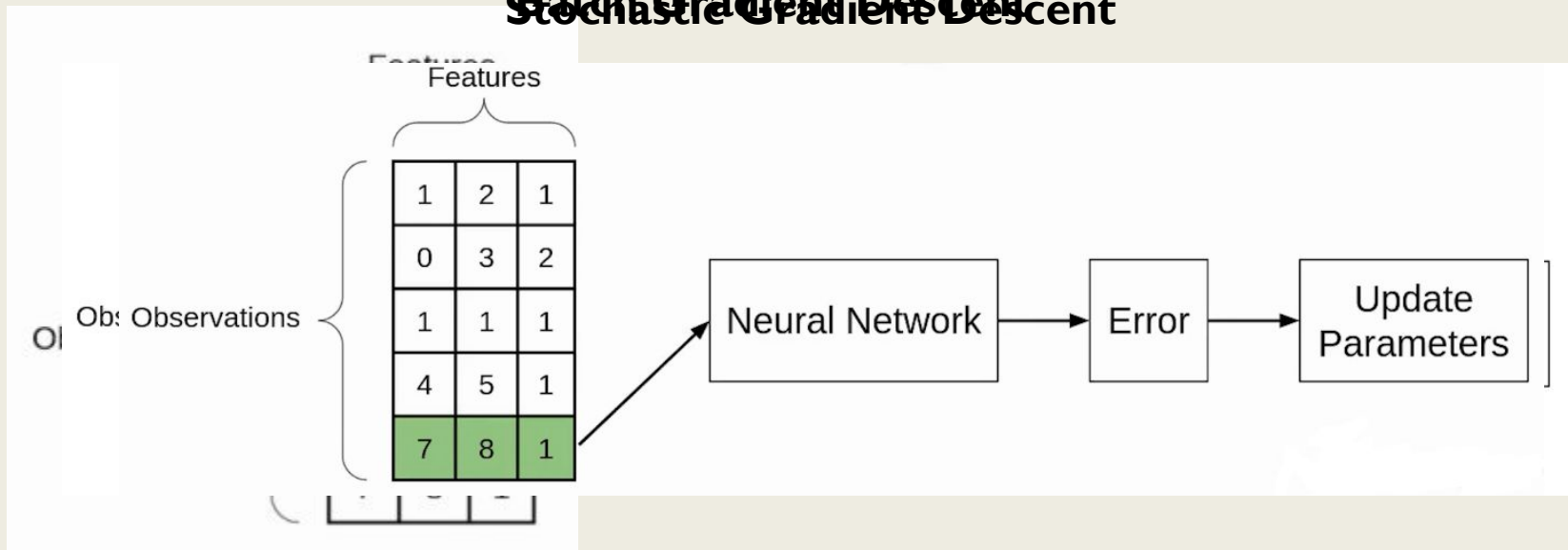
- **Cons**

- Mini-batch requires the configuration of an additional “mini-batch size” *hyperparameter* for the learning algorithm.
- Error information must be accumulated across mini-batches of training examples like batch gradient descent.

Examples

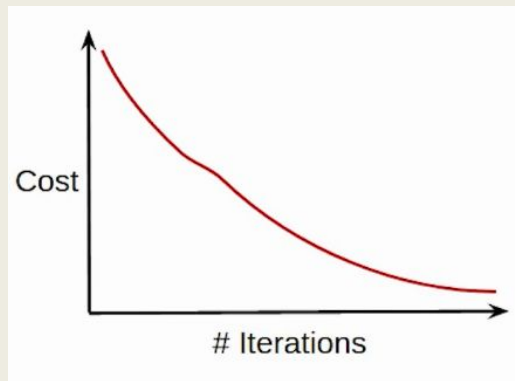
- Let's say we have 5 observations and each observation has three features and the values that I've taken are completely random.

Mini-Batch Gradient Descent
Batch Gradient Descent
Stochastic Gradient Descent

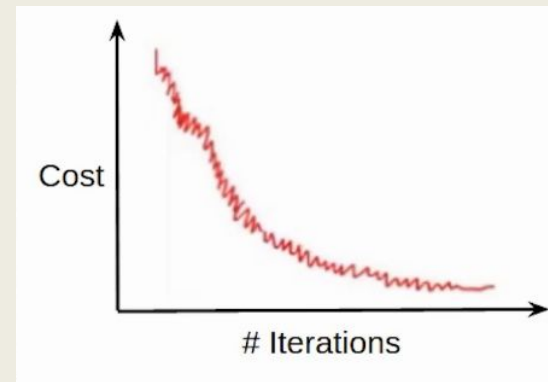


Comparison Chart

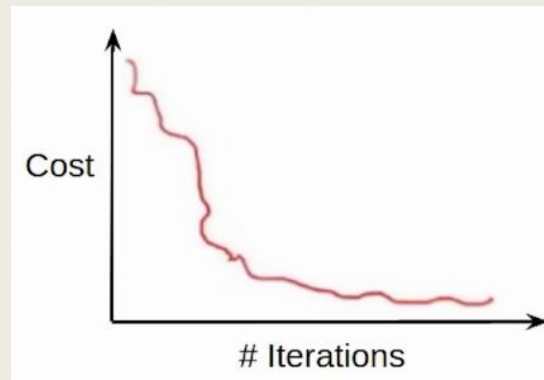
Batch Gradient Descent



Stochastic Gradient Descent



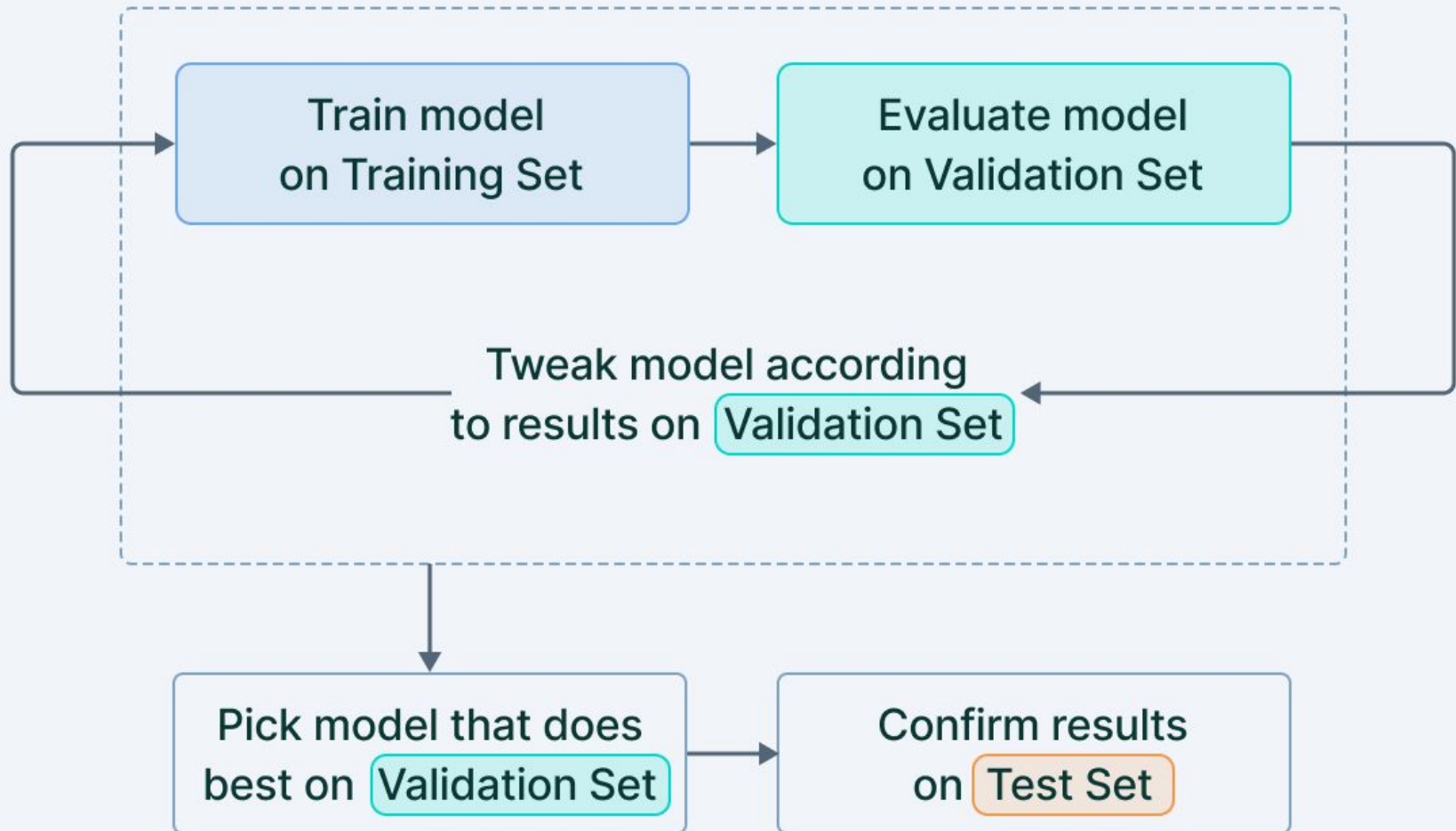
Mini-Batch Gradient Descent



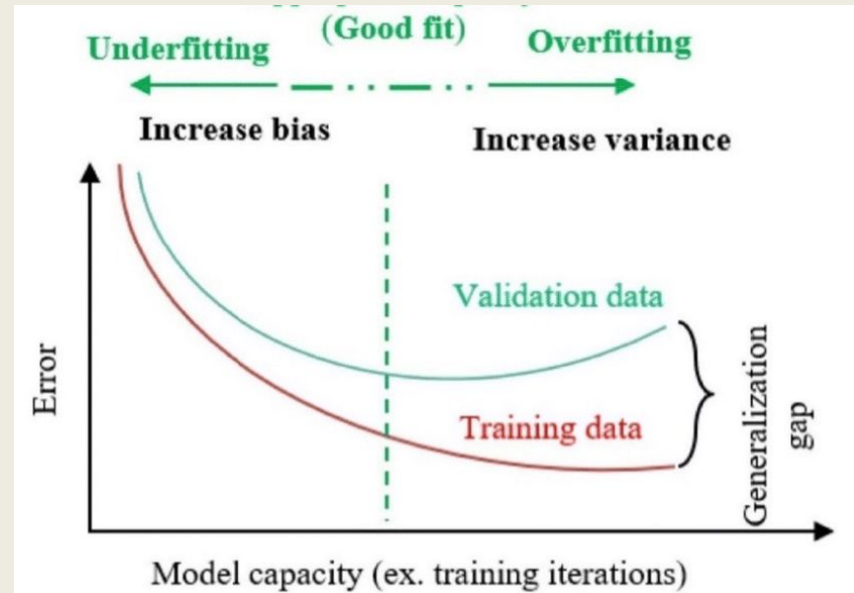
Train, Test and Validation

- **Training Dataset** - The sample of data used to fit the model.
- **Validation Dataset** - The sample of data used to provide an unbiased evaluation of a model fit on the training dataset while tuning model hyperparameters.
- **Test Dataset** - The sample of data used to provide an unbiased evaluation of a final model fit on the training dataset.

Train, Test and Validation



Overfitting and Underfitting



Training Loss	Validation Loss	Situation	Tradeoff
High	Low	Underfitting	High Bias
Low	High	Overfitting	High Variance
Low	Low	Optimal fitting	Low Bias & Low Variance

Overfitting and Underfitting

- Bias – deals with the training data
- Variance – deals with the testing/ Validation data

Error Rate	Model A	Model B	Model C
Training Data	30%	6%	1%
Testing Data	45%	8%	25%

Model A – High Bias and High Variance – Not fit

Model B – Low Bias and Low Variance – Optimal fit

Model C – Low Bias and High Variance – Over fit

Hyper parameters

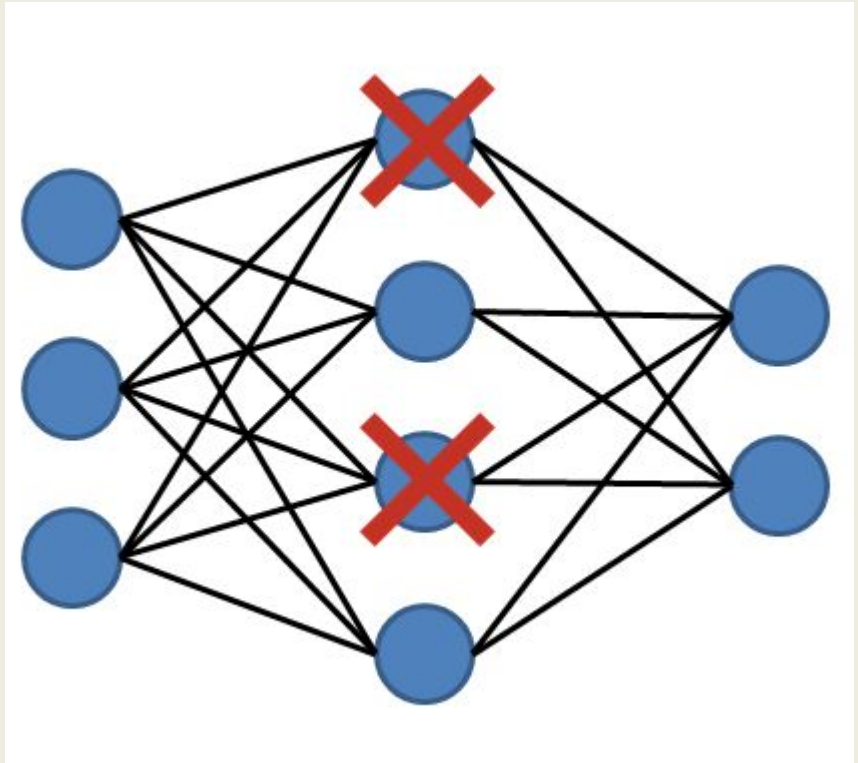
- Determines the
 - Network of the structure (e.g Number of hidden units etc.,)
 - Variables which determine the network is trained (e.g Learning rate)

Hyper parameters related to N/w structure

- Number of Hidden Layers and units – Layers between the input and output layers
- *Keep adding the layers until the test error does not improve anymore*
- *Many hidden units within a layer with regularization techniques can increase accuracy.* Smaller number of units may cause **underfitting**.

Hyper parameters related to N/w structure

- **Dropout** - regularization technique to *avoid overfitting* (increase the validation accuracy)
- Generally, use a small dropout value of 20%-50% of neurons with 20% providing a good starting point.
- Likely to get better performance when dropout is used on a larger network, giving the model more of an opportunity to learn independent representations.



Hyper parameters related to N/w structure

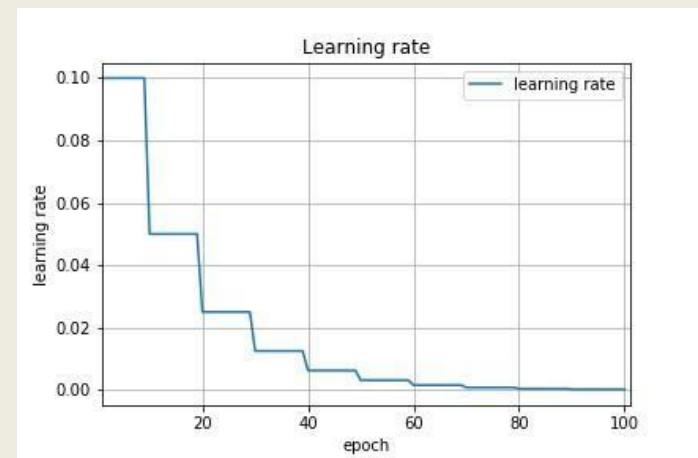
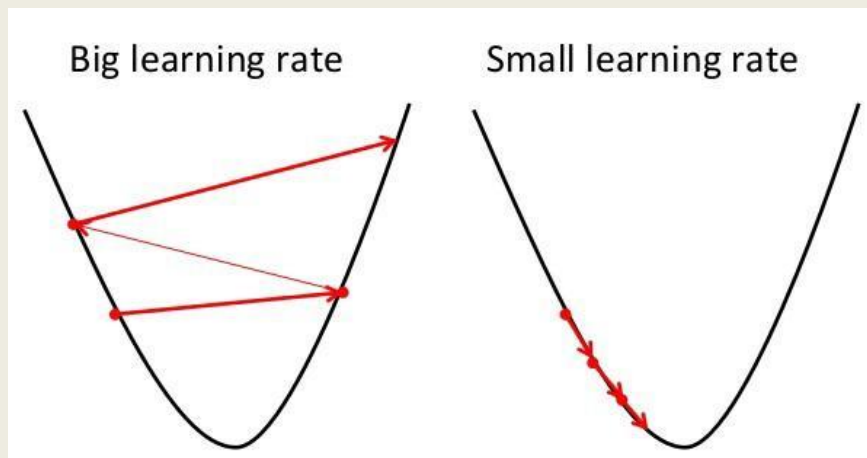
- **Network Weight Initialization**
 - Zero Initialization
 - Random Initialization
 - Xavier/ Glorot Initialization
 - Normalized Xavier/ Glorot Initialization
 - Uniform Distribution
 - He Normal Initialization
- Better to use different weight initialization schemes according to the activation function used on each layer. – Mostly Uniform distribution

Hyper parameters related to N/w structure

- **Activation Functions**
- To **introduce nonlinearity** to models, which allows deep learning models to learn nonlinear prediction boundaries.
- **Sigmoid** is used in the output layer while making **binary predictions**.
- **Softmax** is used in the output layer while making **multi-class predictions**.

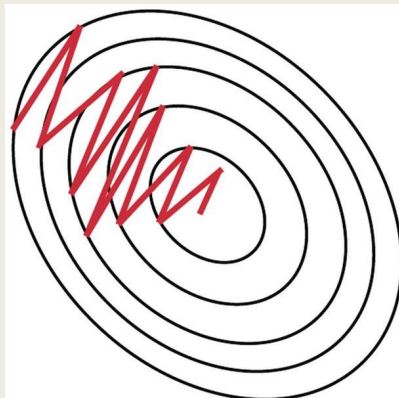
Hyper parameters related to Training Algorithm

- **Learning Rate** - defines how quickly a network updates its parameters. $[10e-4 \text{ to } 1.0]$ - Traditional value – 0.1 or 0.01
- **Low learning rate** slows down the learning process but converges smoothly.
- **Larger learning rate** speeds up the learning but may not converge.
- Usually a **decaying Learning rate** is preferred. – Step, time, exponential

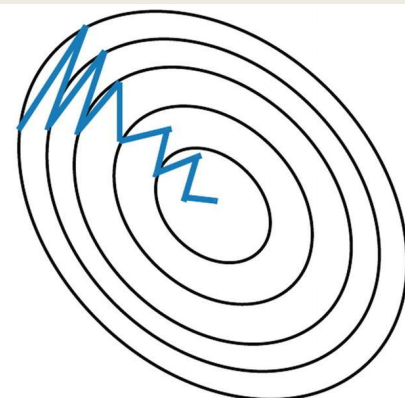


Hyper parameters related to Training Algorithm

- **Momentum**
- Helps to know the direction of the next step with the knowledge of the previous steps.
- It helps to prevent oscillations.
- A typical choice of momentum is between 0.5 to 0.9.



Stochastic Gradient
Descent **without**
Momentum



Stochastic Gradient
Descent **with**
Momentum

Hyper parameters related to Training Algorithm

- **Number of epochs**
- Number of epochs is the number of times the whole training data is shown to the network while training.
- Increase the number of epochs until the validation or training accuracy starts decreasing.

Hyper parameters related to Training Algorithm

- **Batch size**
- Mini batch size is the number of sub samples given to the network after which parameter update happens.
- **A good default for batch size might be 32.**
- Also try 32, 64, 128, 256, and so on.

Hyperparameter Optimization Techniques

- Manual Search
- Grid Search
- Randomized Search
- Halving Grid Search
- Halving Randomized Search
- Hyperopt-sklearn
- Bayes Search
- Evolutionary Algorithms

Thank You